

智慧食堂项目概要设计文档

1. 概述

1.1 项目背景

随着信息技术的快速发展和校园管理精细化需求的提升，传统食堂管理模式在效率、透明度、食品安全追溯等方面面临诸多挑战。为了提升学校食堂的管理水平、保障师生饮食安全与健康、优化食堂运营效率，响应教育主管部门对校园食品安全监管的要求，特启动智慧食堂项目建设。本项目旨在利用现代信息技术手段，构建一个集信息展示、系统管理、基础信息采集、食谱管理、台账管理、采购管理、统计分析及市级数据中心对接等功能于一体的综合性智慧食堂管理平台。平台将覆盖学校食堂运营的各个环节，实现管理的数字化、智能化和规范化，为学校、管理部门、供应商以及师生提供便捷、高效、透明的服务。

1.2 设计目标

本概要设计文档旨在明确智慧食堂项目的整体架构、核心功能模块、关键技术选型、数据库结构、接口规范、安全策略以及部署方案。其主要目标包括：

- 清晰定义系统边界与架构：**明确系统的组成部分、模块划分以及各模块间的交互关系，为后续详细设计和开发提供基础框架。
- 详细描述核心功能实现：**对需求规格说明书中定义的核心功能进行细化设计，说明其实现逻辑、关键流程和主要界面元素，确保功能满足用户需求。
- 确定关键技术方案：**选择合适的技术栈、开发框架、数据库系统等，确保系统的性能、可扩展性、可维护性和安全性。
- 规范数据存储与交互：**设计合理的数据库结构，满足数据存储和查询需求；定义清晰的内外部接口，保障系统间的数据顺畅交互。
- 制定安全与部署策略：**规划系统的安全防护措施，保障数据和系统安全；制定可行的部署方案，确保系统稳定上线运行。
- 作为沟通与评审依据：**为项目团队成员、用户代表及相关干系人提供统一的技术蓝图，作为沟通、评审和验收的重要依据。

1.3 项目范围

本项目范围涵盖智慧食堂管理平台的全部核心功能模块，主要包括：

- 首页展示：**提供移动端登录入口、学期设置、帮助中心、当日食谱、采购信息、台账记录等信息的集中展示。

- **系统管理：** 包括用户管理（子账号、权限）、通知公告管理、食堂风采管理、停餐报备、投诉建议、知识库（法规制度、安全知识）等。
- **基础信息管理：** 涵盖学校食安信息、食堂基本信息、从业人员健康信息、供应商信息、厨余垃圾回收企业信息、管理制度备案等。
- **食谱管理：** 包括食材库管理（内置信息、营养成分）、菜品库管理（配方、用量）、食谱编辑（拖拽式、引用编辑、纠错、分类）等。
- **台账管理：** 支持台账频次配置、移动端便捷记录（含照片上传）、工作提醒与预警、多类型台账（晨检、消毒、加工、留样、陪餐等）、台账下载与统计。
- **采购管理：** 包含基础设置（采购人等）、采购计划（食谱自动生成、手动调整）、线上采购（供应商注册、上架、下单、确认、送货、验收、价格提醒、收藏）、线下采购（入库登记、常用商品库）、票证管理（检验合格证上传）、出库管理（一键生成、选择库存）、库存管理（盘点、查询、统计、批次管理）、结算管理（自动汇总、自定义周期、历史查询）、成本核算（收支记录、自动计算、核算公示）。
- **采购统计：** 自动生成采购台账、入/出库周/月汇总表、历史数据查询、库存明细表、供应商结算表、收支成本统计。
- **综合统计：** 食品安全风险评估与预警、食品溯源信息生成、食堂规范化管理评分。
- **市级数据中心对接：** 实现与市级平台的数据接入，支持信息调阅（基础信息、食谱营养、台账风险、采购监测）。

非本项目范围：硬件设备（如智能秤、支付终端等）的采购与集成、具体的线下食堂运营管理流程优化（平台提供支持，但不直接干预）。

1.4 文档读者对象

本概要设计文档的主要读者对象包括：

- 项目管理人员
- 系统架构师
- 软件开发工程师
- 数据库管理员
- 测试工程师
- 系统运维工程师
- 用户代表及业务部门相关人员
- 项目监理及评审专家

1.5 术语与缩略语

术语/缩略语	全称/解释
SRS	Software Requirements Specification (软件需求规格说明书)
HLD	High-Level Design (概要设计)

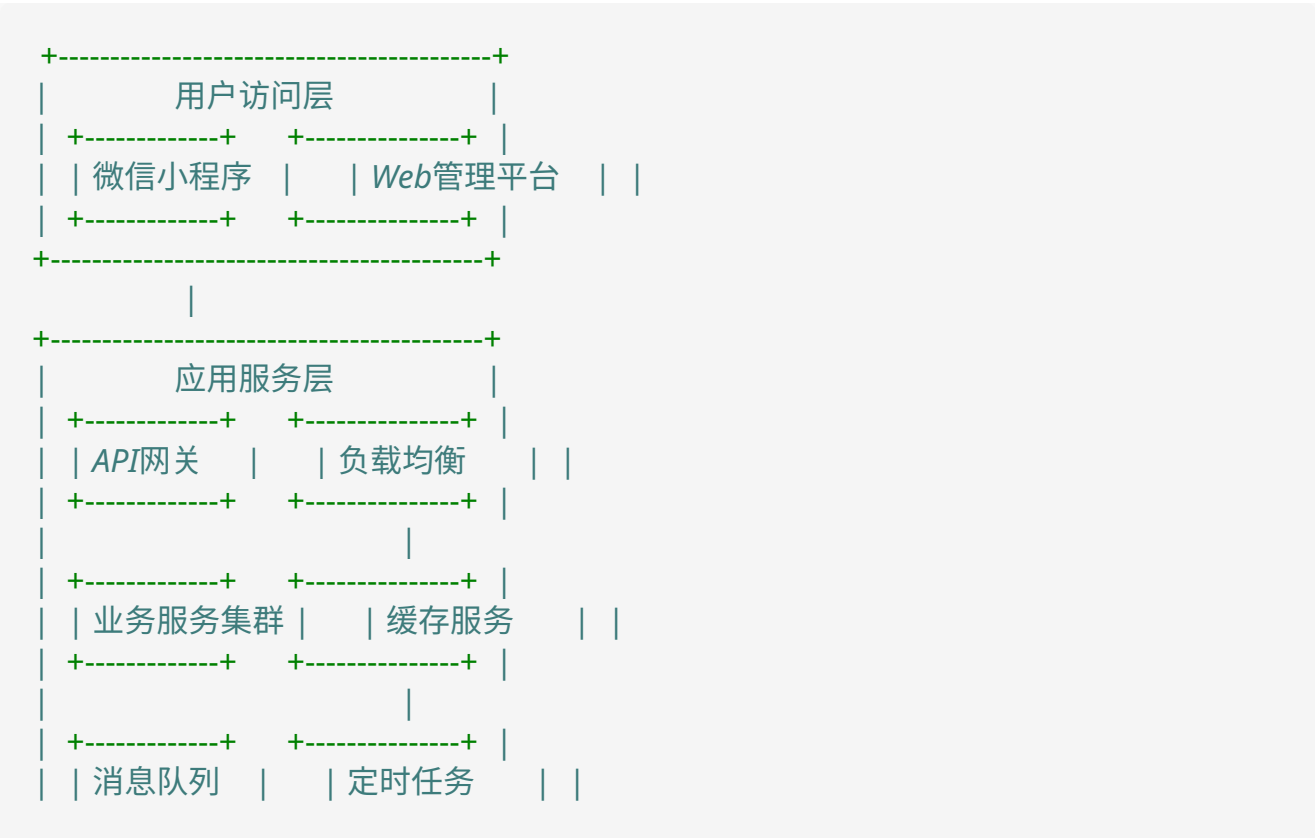
术语/缩略语	全称/解释
UI	User Interface (用户界面)
API	Application Programming Interface (应用程序编程接口)
DB	Database (数据库)
SOP	Standard Operating Procedure (标准操作程序)
RBAC	Role-Based Access Control (基于角色的访问控制)
SSO	Single Sign-On (单点登录)
WXMP	WeChat Mini Program (微信小程序)
SaaS	Software as a Service (软件即服务)

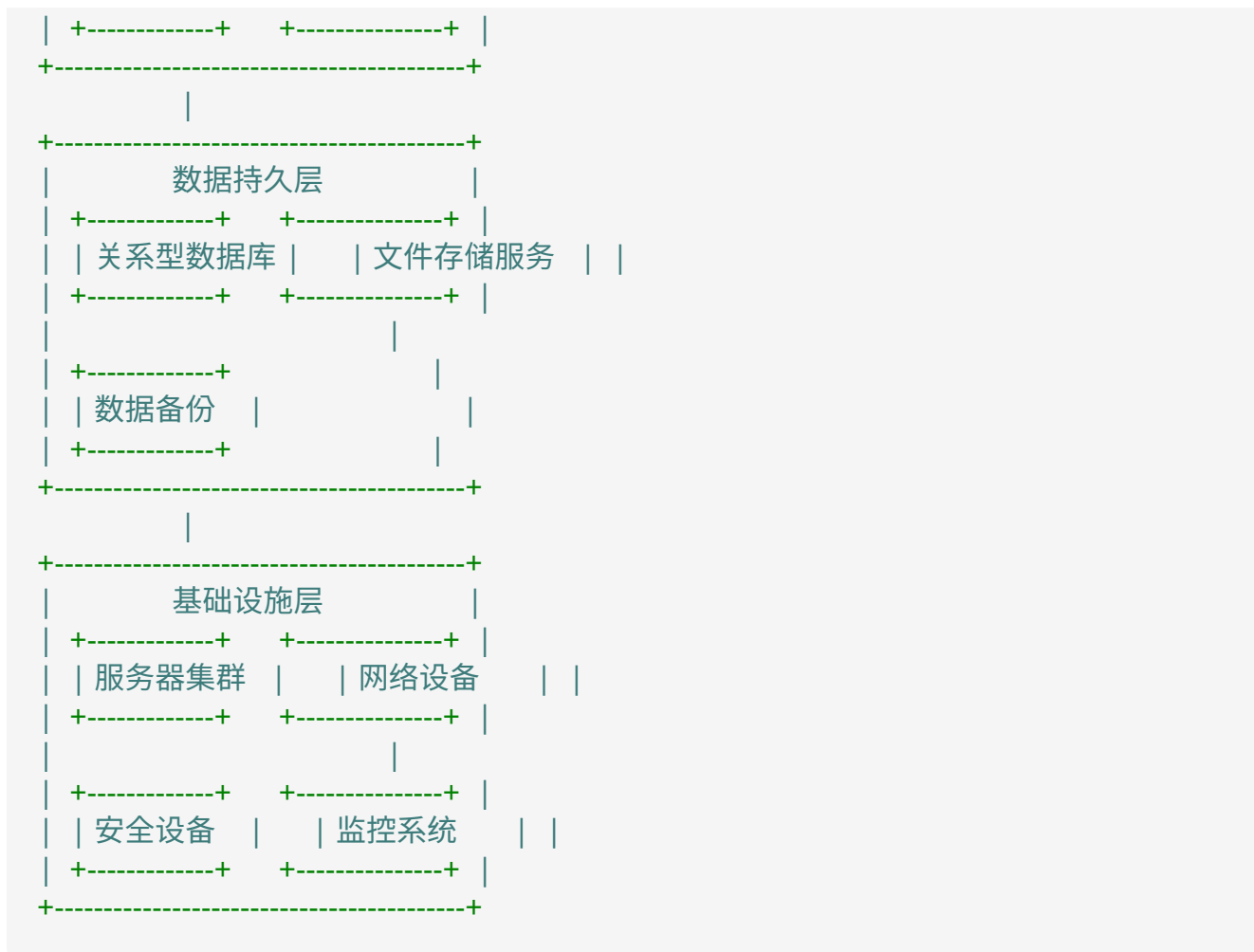
(后续章节中出现的其他专业术语将在首次出现时进行解释)

2. 系统架构设计

2.1 总体架构

智慧食堂项目采用现代化的多层架构设计，以满足系统的可扩展性、可维护性、安全性和性能需求。系统总体架构如下图所示：





2.1.1 用户访问层

用户访问层提供多种终端访问方式，主要包括：

- **微信小程序**：面向学校食堂工作人员、师生等用户，提供移动端便捷操作界面，支持扫码登录、台账记录、食谱查看等日常操作功能。
- **Web管理平台**：面向学校管理员、教育主管部门管理人员、系统管理员等用户，提供完整的管理功能，包括系统配置、数据管理、统计分析等。

2.1.2 应用服务层

应用服务层是系统的核心，负责业务逻辑处理和服务编排，主要组件包括：

- **API网关**：统一的服务入口，负责请求路由、认证授权、限流熔断、协议转换等。
- **负载均衡**：分发请求到多个服务节点，提高系统可用性和性能。
- **业务服务集群**：按功能模块划分的微服务集群，包括用户服务、食谱服务、台账服务、采购服务、统计服务等。
- **缓存服务**：存储热点数据，减轻数据库压力，提高响应速度。
- **消息队列**：处理异步任务，如通知推送、数据同步等。
- **定时任务**：执行周期性任务，如数据统计、报表生成、提醒预警等。

2.1.3 数据持久层

数据持久层负责数据的存储和管理，主要组件包括：

- **关系型数据库**：存储结构化业务数据，如用户信息、食谱信息、台账记录、采购数据等。
- **文件存储服务**：存储非结构化数据，如图片、文档、证照扫描件等。
- **数据备份**：定期备份关键数据，确保数据安全和可恢复性。

2.1.4 基础设施层

基础设施层提供系统运行的硬件和网络环境，主要包括：

- **服务器集群**：提供计算和存储资源。
- **网络设备**：提供网络连接和通信服务。
- **安全设备**：提供网络安全防护，如防火墙、入侵检测等。
- **监控系统**：监控系统运行状态，及时发现和处理异常。

2.2 技术架构

2.2.1 开发技术栈

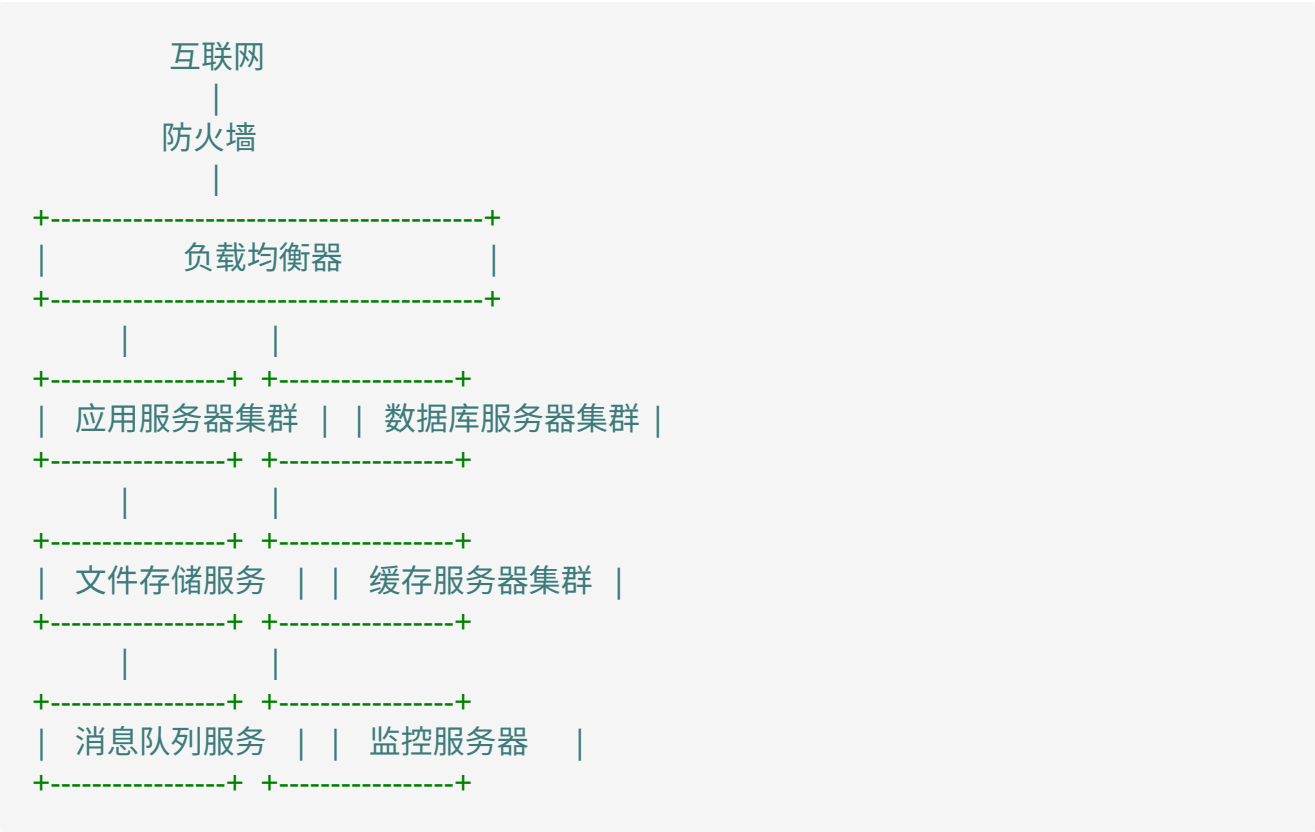
智慧食堂项目采用以下技术栈：

- **前端技术：**
 - Web端：Vue.js + Element UI
 - 移动端：微信小程序框架
 - 数据可视化：ECharts
 - HTTP客户端：Axios
- **后端技术：**
 - 开发语言：Java (Spring Boot)
 - 微服务框架：Spring Cloud
 - API文档：Swagger
 - 安全框架：Spring Security
 - 任务调度：Quartz
- **数据存储：**
 - 关系型数据库：MySQL
 - 缓存：Redis
 - 文件存储：阿里云OSS

- **中间件：**
 - 消息队列：RabbitMQ
 - 搜索引擎：Elasticsearch（用于复杂查询和全文检索）
- **DevOps工具链：**
 - 容器化：Docker
 - 容器编排：Kubernetes
 - CI/CD：Jenkins
 - 代码管理：Git
 - 制品库：Nexus

2.2.2 系统部署架构

系统采用云原生架构，支持在公有云、私有云或混合云环境中部署。主要部署架构如下：



- **生产环境：**采用多节点集群部署，确保高可用性和可扩展性。
- **测试环境：**模拟生产环境配置，但规模较小，用于功能测试和性能测试。
- **开发环境：**单节点部署，用于日常开发和调试。

2.2.3 系统安全架构

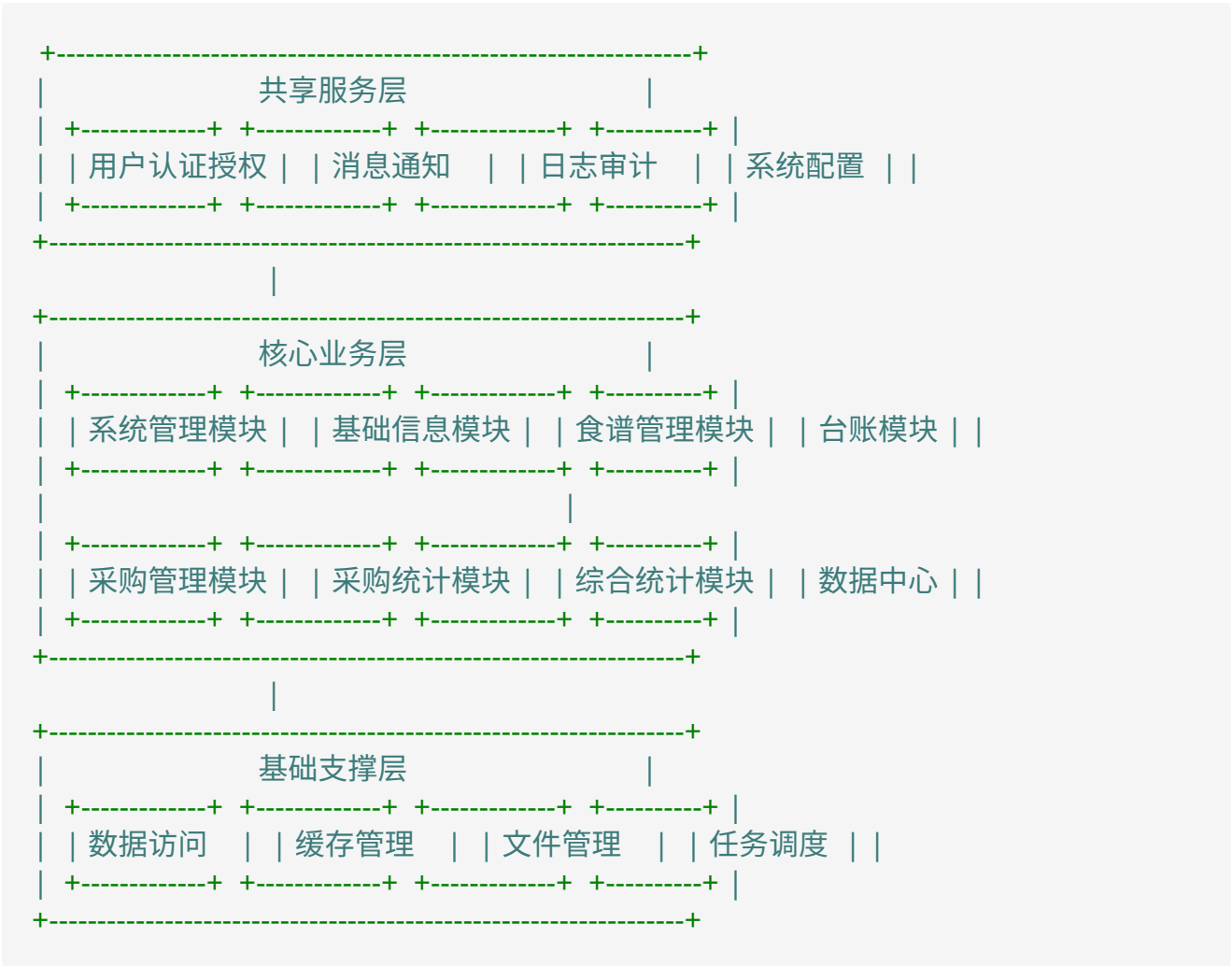
系统安全架构采用纵深防御策略，从多个层面保障系统安全：

- **网络安全：**防火墙、VPN、HTTPS加密传输

- **应用安全**：身份认证、权限控制、输入验证、防XSS/CSRF攻击
- **数据安全**：数据加密、敏感信息脱敏、数据备份与恢复
- **运维安全**：安全审计、日志监控、漏洞扫描

2.3 逻辑架构

系统逻辑架构按照业务领域划分为多个核心模块，各模块之间通过标准接口进行交互：



2.3.1 共享服务层

- **用户认证授权**：提供统一的用户认证和权限控制服务，支持多种认证方式（账号密码、微信扫码等）。
- **消息通知**：提供统一的消息推送服务，支持系统内通知、短信、微信推送等多种方式。
- **日志审计**：记录系统操作日志和业务日志，支持日志查询和审计。
- **系统配置**：管理系统参数配置，支持动态配置和多环境配置。

2.3.2 核心业务层

- **系统管理模块**：用户管理、角色权限、通知公告、食堂风采、停餐报备、投诉建议、知识库等。

- **基础信息模块**：学校食安信息、食堂基本信息、从业人员健康信息、供应商信息、厨余垃圾回收企业信息、管理制度备案等。
- **食谱管理模块**：食材库管理、菜品库管理、食谱编辑等。
- **台账模块**：台账配置、台账记录、台账查询、台账统计等。
- **采购管理模块**：采购计划、线上采购、线下采购、出入库管理、库存管理、结算管理、成本核算等。
- **采购统计模块**：采购台账、入出库汇总、历史数据查询、库存明细、结算表、收支成本统计等。
- **综合统计模块**：食品安全风险评估、食品溯源信息、食堂规范化管理评分等。
- **数据中心模块**：市级数据接入、信息调阅、数据分析等。

2.3.3 基础支撑层

- **数据访问**：提供统一的数据访问接口，封装数据库操作细节。
- **缓存管理**：管理系统缓存，提高数据访问效率。
- **文件管理**：处理文件上传下载、存储管理等。
- **任务调度**：管理定时任务和异步任务的执行。

2.4 数据流架构

系统主要数据流如下：

2.4.1 用户认证数据流

用户 -> 登录请求 -> 认证服务 -> 验证凭证 -> 生成令牌 -> 返回用户

2.4.2 食谱管理数据流

学校管理员 -> 编辑食谱 -> 食谱服务 -> 保存食谱 -> 数据库
-> 生成营养分析 -> 展示结果

2.4.3 台账记录数据流

食堂工作人员 -> 记录台账 -> 台账服务 -> 验证数据 -> 保存记录 -> 数据库
-> 更新统计 -> 生成提醒

2.4.4 采购管理数据流

食堂管理员 -> 创建采购计划 -> 采购服务 -> 生成采购单 -> 数据库
供应商 -> 确认订单 -> 采购服务 -> 更新订单状态 -> 数据库

食堂验收员 -> 验收入库 -> 库存服务 -> 更新库存 -> 数据库
-> 生成入库单 -> 数据库

2.4.5 数据中心数据流

市级平台 -> 数据请求 -> 数据中心服务 -> 查询数据 -> 数据库
-> 处理数据 -> 返回结果

3. 功能模块设计

3.1 首页模块

3.1.1 功能描述

首页模块是系统的入口，为用户提供系统概览和快捷访问功能，集中展示当日关键信息和常用功能入口。

3.1.2 功能组件

1. 移动端登录入口

2. 功能：提供微信小程序扫码登录功能

3. 实现要点：

- 生成唯一二维码
- 微信授权认证
- 用户身份绑定与验证
- 登录状态维护

4. 学期设置

5. 功能：设置学期开始和结束时间，作为统计数据的时间范围基准

6. 实现要点：

- 学期时间配置界面
- 时间选择控件
- 配置保存与应用
- 历史学期记录管理

7. 系统帮助

8. 功能：提供系统使用帮助和常见问题解决方案

9. 实现要点：

- 帮助内容分类展示
- 搜索功能
- 常见问题FAQ
- 操作指南与教程

10. 当日食谱展示

11. 功能：展示当日食谱信息

12. 实现要点：

- 按餐次分类展示
- 菜品详情查看
- 营养成分展示
- 食谱打印功能

13. 当日采购信息

14. 功能：展示学校食堂当日食材采购信息

15. 实现要点：

- 采购品类汇总
- 采购数量与金额
- 供应商信息
- 采购状态跟踪

16. 当日台账记录

17. 功能：展示食堂当日台账记录情况

18. 实现要点：

- 台账完成情况概览
- 未完成台账提醒
- 台账记录快速入口
- 台账详情查看

3.1.3 界面设计

首页界面采用卡片式布局，主要包括以下区域：

1. **顶部导航栏**：系统logo、用户信息、消息通知、帮助入口
2. **功能快捷入口**：常用功能的图标式快捷入口
3. **当日概览**：食谱、采购、台账等当日关键信息的概览卡片
4. **通知公告**：最新通知公告滚动展示

- 5. **工作提醒**：待办事项和工作提醒
- 6. **底部导航**：在移动端提供主要功能的快捷导航

3.1.4 关键流程

1. 用户登录流程：

用户 -> 扫描二维码 -> 微信授权 -> 系统验证身份 -> 生成会话令牌 -> 登录成功 -> 进入首页

2. 学期设置流程：

管理员 -> 进入学期设置 -> 选择开始/结束日期 -> 保存配置 -> 系统应用新设置 -> 更新统计基准

3. 当日信息加载流程：

用户进入首页 -> 系统获取当前日期 -> 并行查询食谱/采购/台账数据 -> 数据聚合 -> 渲染展示

3.2 系统管理模块

3.2.1 功能描述

系统管理模块负责平台的基础管理功能，包括用户管理、通知公告、食堂风采、停餐报备、投诉建议和知识库等，为平台的正常运行提供支持。

3.2.2 功能组件

1. 通知公告管理

2. 功能：展示、查看管理部门下发的通知公告信息

3. 实现要点：

- 通知分类管理
- 通知发布与审核流程
- 通知查看状态跟踪
- 通知搜索与归档

4. 食堂风采管理

5. 功能：展示、查看学校食堂风采

6. 实现要点：

- 风采内容编辑
- 图片上传与管理
- 审核与发布流程

- 风采展示页面

7. 子账号管理

8. 功能：新增、删除子账号，为子账号设置权限

9. 实现要点：

- 账号信息管理
- 角色与权限配置
- 账号状态管理
- 操作日志记录

10. 通知公告管理（管理部门）

11. 功能：管理部门编辑、修改、删除、查看、发布通知公告，学校查看通知公告

12. 实现要点：

- 通知编辑器
- 通知发布范围设置
- 通知状态管理
- 通知阅读统计

13. 食堂风采管理（学校与管理部门）

14. 功能：学校编辑、提交本校的食堂风采，管理部门对学校的食堂风采审核、发布、查看

15. 实现要点：

- 风采内容模板
- 审核流程管理
- 发布状态控制
- 展示效果预览

16. 停餐报备

17. 功能：学校停餐报备，管理部门统计、查看学校停餐信息

18. 实现要点：

- 报备表单设计
- 报备审批流程
- 报备记录管理
- 统计分析功能

19. 投诉建议

20. 功能：学校对供应商提出投诉与建议，管理部门可查看、统计对供应商的投诉与建议

21. 实现要点：

- 投诉表单设计
- 投诉处理流程
- 投诉状态跟踪
- 投诉统计分析

22. 知识库

23. 功能：包括学校食堂经营相关的法律法规，食堂管理制度，食堂各岗位制度，食品安全知识、营养健康知识等

24. 实现要点：

- 知识分类管理
- 内容编辑与发布
- 搜索与标签功能
- 阅读记录与统计

3.2.3 界面设计

系统管理模块界面采用左侧导航 + 右侧内容区的经典布局：

1. **左侧导航**：功能分类菜单，包括通知公告、食堂风采、账号管理等
2. **右侧内容区**：根据选择的功能显示相应的操作界面
3. **列表页**：采用表格形式展示数据，支持搜索、筛选、排序等操作
4. **详情页**：采用表单形式展示和编辑详细信息
5. **操作区**：提供新增、编辑、删除等操作按钮

3.2.4 关键流程

1. 通知公告发布流程：

管理部门 -> 创建通知 -> 编辑内容 -> 设置发布范围 -> 提交审核 -> 审核通过 -> 发布通知 -> 学校接收通知

2. 食堂风采审核流程：

学校 -> 编辑风采 -> 提交审核 -> 管理部门审核 -> 审核通过/驳回 -> 通过则发布展示/驳回则返回修改

3. 子账号权限设置流程：

管理员 -> 创建子账号 -> 设置基本信息 -> 分配角色 -> 设置具体权限 -> 保存设置 -> 子账号生效

3.3 基础信息模块

3.3.1 功能描述

基础信息模块负责采集和管理学校食堂的各类基础信息，包括食安信息、食堂基本信息、从业人员信息、供应商信息、厨余垃圾回收企业信息和管理制度备案等，为系统其他功能模块提供数据支持。

3.3.2 功能组件

1. 食品安全信息管理

2. 功能：采集学校食品安全信息相关内容，管理部门可汇总、查询学校食安信息

3. 实现要点：

- 安全信息分类采集
- 信息审核与确认
- 信息汇总与查询
- 安全风险评估

4. 食堂基本信息管理

5. 功能：采集学校食堂基本信息，包括相关证照，管理部门对学校食安信息汇总、查询

6. 实现要点：

- 基本信息表单设计
- 证照上传与管理
- 信息变更记录
- 证照有效期提醒

7. 从业人员健康管理信息

8. 功能：采集学校食堂从业人员健康信息，管理部门对学校从业人员信息汇总、查询，健康证临期提醒

9. 实现要点：

- 人员信息管理
- 健康证信息录入
- 临期自动提醒
- 人员变动管理

10. 供应商信息管理

11. 功能：采集学校食材供应商信息，包供相关证照。管理部门对学校供应商信息汇总、查询

12. 实现要点：

- 供应商基本信息管理
- 供应商资质审核
- 供应商分类管理
- 供应商评价体系

13. 厨余垃圾回收企业信息管理

14. 功能：采集学校食堂厨余垃圾回收企业信息，包括相关证照信息，管理部门对学校收储企业信息汇总、查询

15. 实现要点：

- 回收企业信息管理
- 证照有效期管理
- 回收记录管理
- 合规性检查

16. 管理制度备案

17. 功能：对学校的食品安全管理制度，食品安全事故应急预案、食堂各岗位管理制度等相关食品安全管理制度、规定上传备案，以备查询、追溯

18. 实现要点：

- 制度文档上传
- 文档分类管理
- 版本控制
- 查询与下载

3.3.3 界面设计

基础信息模块界面设计注重信息的清晰展示和便捷操作：

1. **信息概览页**：展示各类基础信息的汇总情况和提醒事项
2. **信息列表页**：采用表格形式展示各类信息列表，支持搜索、筛选、排序
3. **信息详情页**：采用分组表单形式展示和编辑详细信息
4. **证照管理页**：提供证照上传、预览、下载等功能
5. **提醒设置页**：配置各类临期提醒规则

3.3.4 关键流程

1. 从业人员健康证管理流程：

管理员 -> 录入人员信息 -> 上传健康证 -> 设置有效期 -> 系统记录 -> 临期自动提醒 -> 更新健康证

2. 供应商信息管理流程：

管理员 -> 录入供应商信息 -> 上传资质证照 -> 审核确认 -> 供应商启用 -> 定期复核资质

3. 管理制度备案流程：

学校 -> 编制管理制度 -> 上传制度文档 -> 填写备案信息 -> 提交备案 -> 管理部门审核 -> 备案完成

3.4 食谱管理模块

3.4.1 功能描述

食谱管理模块负责食材库、菜品库的管理以及食谱的编制、发布和营养分析，为学校食堂提供科学、合理的膳食安排支持。

3.4.2 功能组件

1. 食材库管理

2. 功能：包括食材品名管理与分类管理，食材营养成份管理。内置4000种以上常用食材的相关信息

3. 实现要点：

- 食材分类体系
- 食材基本信息管理
- 营养成分数据库
- 食材搜索与筛选

4. 菜品库管理

5. 功能：主要包括菜品名称、配方（即该菜品用到哪些食材）信息；菜品对应不同用餐人群的食材用量信息

6. 实现要点：

- 菜品基本信息管理
- 配方管理（食材组成）
- 用量标准管理
- 菜品图片管理

7. 食谱编辑

8. 功能：提供多种食谱编辑方式，包括直接编辑、引用食谱二次编辑、食谱纠错、菜品分类等

9. 实现要点：

- 拖拽式编辑界面
- 食谱模板与引用
- 编辑数据自动保存
- 菜品分类管理

3.4.3 界面设计

食谱管理模块界面设计注重操作便捷性和视觉直观性：

1. **食材库管理界面：**

2. 左侧食材分类树

3. 右侧食材列表（表格形式）

4. 食材详情弹窗（包含基本信息和营养成分）

5. **菜品库管理界面：**

6. 菜品分类导航

7. 菜品列表（卡片形式，显示菜品图片和名称）

8. 菜品详情页（包含基本信息、配方、用量标准）

9. **食谱编辑界面：**

10. 左侧菜品选择区（分类展示）

11. 右侧食谱编排区（按餐次和日期组织的表格）

12. 拖拽操作区（支持菜品拖拽到食谱表格中）

13. 食谱预览区

3.4.4 关键流程

1. **食谱编辑流程：**

管理员 -> 选择日期 -> 创建/编辑食谱 -> 从菜品库选择菜品 -> 拖拽到对应餐次 -> 调整用量 -> 保存食谱 -> 发布食谱

2. **食谱引用流程：**

管理员 -> 查询历史食谱 -> 选择要引用的食谱 -> 复制到当前编辑区 -> 调整修改 -> 保存为新食谱

3. 营养分析流程：

食谱保存 -> 系统自动计算营养成分 -> 生成营养分析报告 -> 与标准对比 -> 提供优化建议

3.5 台账管理模块

3.5.1 功能描述

台账管理模块负责食堂各类台账的配置、记录、查询和统计，通过规范化的台账管理，提升食堂日常运营的规范性和可追溯性。

3.5.2 功能组件

1. 台账配置

2. 功能：管理部门配置学校各类台账记录频次，学校配置台账相关基础信息

3. 实现要点：

- 台账类型管理
- 记录频次设置
- 台账模板配置
- 基础信息预设

4. 台账记录

5. 功能：支持食堂工作人员便捷记录台账，包括移动端操作和照片上传

6. 实现要点：

- 移动端适配
- 表单智能填充
- 照片上传与压缩
- 离线记录与同步

7. 工作提醒

8. 功能：系统自动提醒每个人当日工作，对未完成工作项目进行警示

9. 实现要点：

- 提醒规则配置
- 多渠道提醒（系统内、短信、微信）
- 提醒状态跟踪
- 提醒升级机制

10. 台账类型管理

11. 功能：支持多种台账类型，包括人员晨检、设施设备消毒、食品加工、留样、陪餐评价等

12. 实现要点：

- 台账分类管理
- 台账模板定制
- 必填项设置
- 台账关联配置

13. 台账下载

14. 功能：可按时间段、台账类别下载台账

15. 实现要点：

- 多格式导出（Excel、PDF）
- 批量下载
- 自定义导出内容
- 导出权限控制

16. 台账统计

17. 功能：统计某个时间段内各类台账应做数量，实做数量，缺做数量，并可查询具体缺失台账

18. 实现要点：

- 统计维度配置
- 图表可视化展示
- 异常数据标记
- 统计报表导出

3.5.3 界面设计

台账管理模块界面设计注重操作简便性和信息直观性：

1. 台账配置界面：

2. 台账类型列表

3. 频次设置表单

4. 模板编辑器

5. 台账记录界面：

6. 待记录台账提示

7. 简化的记录表单

8. 照片上传区

9. 快速提交按钮

10. 台账查询界面：

11. 多条件筛选区

12. 台账列表（表格形式）

13. 台账详情查看

14. 批量操作区

15. 台账统计界面：

16. 统计图表区（柱状图、饼图等）

17. 统计数据表格

18. 异常数据标记

19. 导出按钮

3.5.4 关键流程

1. 台账记录流程：

工作人员 -> 接收提醒 -> 打开记录界面 -> 填写台账内容 -> 上传相关照片 -> 提交记录 -> 系统验证 -> 记录保存

2. 台账统计分析流程：

管理员 -> 设置统计条件 -> 系统查询数据 -> 计算统计指标 -> 生成统计图表 -> 标记异常数据 -> 展示统计结果

3. 台账提醒流程：

系统定时任务 -> 检查当日应做台账 -> 生成提醒任务 -> 推送提醒消息 -> 跟踪完成状态 -> 未完成则再次提醒

3.6 采购管理模块

3.6.1 功能描述

采购管理模块负责食堂食材的采购计划、订单管理、验收入库、出库管理、库存管理、结算管理和成本核算等，实现食材采购全流程的规范化管理。

3.6.2 功能组件

1. 基础设置

2. 功能：配置采购人、验收人、保管员等信息

3. 实现要点：

- 人员信息管理
- 角色权限设置
- 审批流程配置
- 基础参数设置

4. 采购计划

5. 功能：系统根据食谱自动生成采购计划，支持手动调整

6. 实现要点：

- 食谱关联分析
- 库存智能检查
- 计划自动生成
- 手动调整界面

7. 线上采购

8. 功能：支持供应商账号注册、食材上架、学校下单、供应商确认等完整流程

9. 实现要点：

- 供应商门户
- 商品管理系统
- 订单处理流程
- 价格调整机制

10. 线下采购

11. 功能：支持学校按原有采购模式操作，仅在系统中记录入库信息

12. 实现要点：

- 简化的入库登记
- 常用商品库
- 供应商快速选择
- 批量录入功能

13. 票证管理

14. 功能：管理食材检验合格证等票证信息

15. 实现要点：

- 票证上传与识别

- 票证与商品关联
- 票证有效性验证
- 票证查询与追溯

16. 出库管理

17. 功能：支持多种出库单生成方式，包括根据入库单、食谱或库存生成

18. 实现要点：

- 出库单自动生成
- 出库数量调整
- 出库记录管理
- 出库单打印

19. 库存管理

20. 功能：支持库存盘点、查询、统计和批次管理

21. 实现要点：

- 实时库存监控
- 库存预警机制
- 批次追溯管理
- 库存盘点工具

22. 结算管理

23. 功能：自动汇总未结算采购单，支持自定义结算周期

24. 实现要点：

- 结算单自动生成
- 结算周期设置
- 结算审核流程
- 结算记录查询

25. 成本核算

26. 功能：记录餐费收入和各类成本支出，自动生成成本核算公示

27. 实现要点：

- 收入记录管理
- 成本分类统计
- 核算公示生成
- 成本分析报表

3.6.3 界面设计

采购管理模块界面设计注重流程清晰和操作便捷：

1. 采购计划界面：

2. 计划列表（表格形式）

3. 计划详情编辑

4. 自动生成按钮

5. 调整确认区

6. 采购订单界面：

7. 订单创建表单

8. 商品选择区

9. 供应商选择区

10. 订单状态跟踪

11. 入库管理界面：

12. 待入库订单列表

13. 入库登记表单

14. 票证上传区

15. 批量操作区

16. 库存管理界面：

17. 库存概览（图表形式）

18. 库存明细列表

19. 预警信息提示

20. 批次管理入口

21. 成本核算界面：

22. 收入记录表单

23. 支出分类统计

24. 成本分析图表

25. 核算公示预览

3.6.4 关键流程

1. 采购计划生成流程：

系统定时任务 -> 分析未来食谱 -> 计算所需食材 -> 检查现有库存 -> 生成采购建议 -> 管理员审核调整 -> 确认采购计划

2. 线上采购流程：

学校 -> 创建采购单 -> 选择供应商和商品 -> 提交订单 -> 供应商确认 -> 供应商发货 -> 学校验收 -> 系统入库 -> 生成入库单

3. 库存管理流程：

入库操作 -> 更新库存 -> 系统监控库存水平 -> 低于阈值则预警 -> 管理员定期盘点 -> 调整实际库存 -> 记录盘点差异

4. 成本核算流程：

记录收入 -> 自动汇总支出 -> 系统计算成本指标 -> 生成核算报表 -> 管理员审核 -> 提交公示 -> 锁定数据

3.7 采购统计模块

3.7.1 功能描述

采购统计模块负责对采购数据进行汇总、分析和展示，为学校和管理部门提供采购管理的决策支持。

3.7.2 功能组件

1. 采购台账

2. 功能：自动生成学校食材采购台账，管理部门可在线查阅管理

3. 实现要点：

- 台账自动生成
- 多维度查询
- 数据导出功能
- 权限控制机制

4. 入库汇总

5. 功能：自动生成学校食材采购入库的周汇总表、月汇总表

6. 实现要点：

- 时间维度汇总
- 品类维度汇总

- 供应商维度汇总
- 图表可视化展示

7. 出库汇总

8. 功能：自动生成学校食材采购出库的周汇总表、月汇总表

9. 实现要点：

- 时间维度汇总
- 品类维度汇总
- 用途维度汇总
- 图表可视化展示

10. 历史数据查询

11. 功能：根据日期、品名、供应商等多条件查询学校历史采购数据

12. 实现要点：

- 多条件组合查询
- 查询结果排序
- 数据导出功能
- 查询性能优化

13. 库存明细

14. 功能：自动生成学校库存食材明细表

15. 实现要点：

- 实时库存查询
- 批次信息展示
- 库存预警标记
- 库存趋势分析

16. 结算表生成

17. 功能：自动按食材分类生成每个供应商的结算表

18. 实现要点：

- 结算周期设置
- 分类汇总计算
- 结算表格式定制
- 结算记录管理

19. 成本管理

20. 功能：包括食堂收入管理、支出成本管理、成本核算公示

21. 实现要点：

- 收支记录管理
- 成本分析计算
- 核算指标设置
- 公示报表生成

3.7.3 界面设计

采购统计模块界面设计注重数据的直观展示和分析功能：

1. **统计概览界面：**

2. 关键指标卡片

3. 趋势图表区

4. 异常数据提示

5. 快速筛选按钮

6. **详细报表界面：**

7. 多维度筛选区

8. 数据表格（支持排序、筛选）

9. 图表展示区

10. 导出打印区

11. **分析工具界面：**

12. 对比分析工具

13. 趋势分析工具

14. 异常分析工具

15. 预测分析工具

3.7.4 关键流程

1. **采购台账生成流程：**

系统定时任务 -> 汇总采购数据 -> 按规则生成台账 -> 存储台账记录 -> 提供查询接口

2. **多维度分析流程：**

用户 -> 设置分析维度 -> 选择时间范围 -> 系统查询数据 -> 执行统计计算 -> 生成分析结果 -> 展示图表和数据

3. 结算表生成流程：

用户 -> 选择结算周期 -> 选择供应商 -> 系统汇总数据 -> 按分类计算金额 -> 生成结算表 -> 提供打印/导出

3.8 综合统计模块

3.8.1 功能描述

综合统计模块负责对食品安全风险、食品溯源和食堂管理规范化等方面进行综合评估和分析，为食堂管理提供决策支持。

3.8.2 功能组件

1. 食品安全风险评估

2. 功能：根据证照备案、管理制度、日常工作完成情况等生成食品安全风险值

3. 实现要点：

- 风险评估模型
- 多维度数据采集
- 风险等级划分
- 预警机制设计

4. 食品溯源信息

5. 功能：自动生成每日学生所吃食品的完整溯源信息

6. 实现要点：

- 溯源链路构建
- 关键节点记录
- 溯源码生成
- 溯源信息查询

7. 规范化管理评分

8. 功能：根据食谱公示、营养分析、台账记录等多维度自动评分

9. 实现要点：

- 评分指标体系
- 自动评分算法
- 评分结果展示
- 改进建议生成

3.8.3 界面设计

综合统计模块界面设计注重数据可视化和直观展示：

- 1. **风险评估界面：**
 - 2. 风险仪表盘
 - 3. 风险因素分析图
 - 4. 历史趋势图
 - 5. 预警信息区
- 6. **食品溯源界面：**
 - 7. 溯源链路图
 - 8. 关键节点信息卡
 - 9. 溯源码查询工具
 - 10. 溯源报告生成
- 11. **规范化评分界面：**
 - 12. 评分雷达图
 - 13. 各维度得分明细
 - 14. 同类学校对比图
 - 15. 改进建议区

3.8.4 关键流程

1. 风险评估流程：

系统定时任务 -> 采集多维度数据 -> 应用风险评估模型 -> 计算风险值 -> 确定风险等级 -> 生成风险报告 -> 触发预警（如需）

2. 食品溯源流程：

食品制作完成 -> 记录制作信息 -> 关联原料信息 -> 关联供应商信息 -> 生成溯源链路 -> 生成溯源码 -> 提供溯源查询

3. 规范化评分流程：

系统定期评估 -> 收集各维度数据 -> 应用评分规则 -> 计算各项得分 -> 生成总体评分 -> 分析不足项 -> 提供改进建议

3.9 市级数据中心模块

3.9.1 功能描述

市级数据中心模块负责与市级智慧食堂平台对接，实现数据的汇聚、共享和分析，为教育主管部门提供全市学校食堂的监管支持。

3.9.2 功能组件

1. 数据接入

2. 功能：将长沙市县区已建设"智慧食堂"平台的数据完全接入

3. 实现要点：

- 数据接口标准化
- 数据同步机制
- 数据质量控制
- 接入状态监控

4. 食谱营养分析

5. 功能：调阅学校食谱的营养评价分析，并进行统计分析

6. 实现要点：

- 营养标准库
- 评价指标体系
- 统计分析模型
- 结果可视化展示

7. 台账风险分析

8. 功能：通过学校台账记录数据，分析判别学校食堂的食品安全管理风险

9. 实现要点：

- 风险模型构建
- 异常模式识别
- 风险等级划分
- 风险地图展示

10. 采购监测

11. 功能：调阅学校采购台账、相关票证，对数据进行统计、分析，实现对学校食材采购价格、金额监测

12. 实现要点：

- 价格监测模型
- 异常交易识别
- 价格趋势分析
- 预算执行监控

3.9.3 界面设计

市级数据中心模块界面设计注重宏观展示和深度分析：

1. 数据概览界面：

2. 学校分布地图
3. 关键指标总览
4. 异常情况提示

5. 快速筛选工具

6. 分析工具界面：

7. 多维度分析工具
8. 对比分析工具
9. 趋势分析工具

10. 报表生成工具

11. 监管支持界面：

12. 风险学校标记
13. 检查任务生成
14. 整改跟踪管理
15. 效果评估工具

3.9.4 关键流程

1. 数据接入流程：

县区平台 -> 数据接口对接 -> 数据格式转换 -> 数据质量检查 -> 数据入库 -> 数据更新
监控

2. 综合分析流程：

用户 -> 选择分析维度 -> 设置分析条件 -> 系统查询数据 -> 执行分析模型 -> 生成分析
结果 -> 展示分析报告

3. 监管支持流程：

系统监测数据 -> 发现异常情况 -> 生成风险提示 -> 推送给监管人员 -> 生成检查任务 -> 跟踪整改情况 -> 评估整改效果

4. 数据库设计

4.1 数据库选型

考虑到系统的业务复杂性、数据一致性要求以及技术团队的熟悉程度，本项目选用 **MySQL** 作为主要的业务关系型数据库。MySQL 具有成熟稳定、性能良好、社区支持广泛等优点，能够满足智慧食堂项目的需求。

对于缓存需求，选用 **Redis**，用于存储热点数据、会话信息等，提高系统响应速度。

对于文件存储，选用 **阿里云OSS**，用于存储图片、文档、证照等非结构化数据，提供高可用、高可靠的存储服务。

4.2 概念模型设计 (E-R 图)

系统的核心实体及其关系如下：

```
erDiagram
    USER ||--o{ ROLE : has
    ROLE ||--|{ PERMISSION : has
    USER ||--o{ SCHOOL : belongs_to
    SCHOOL ||--o{ CANTEEN : has
    CANTEEN ||--|{ EMPLOYEE : employs
    EMPLOYEE ||--o{ HEALTH_CERT : has
    CANTEEN ||--|{ SUPPLIER : collaborates_with
    SUPPLIER ||--|{ INGREDIENT : supplies
    CANTEEN ||--|{ WASTE_RECYCLER : collaborates_with
    CANTEEN ||--|{ CANTEEN_INFO : has
    CANTEEN_INFO ||--|{ LICENSE : includes
    CANTEEN ||--|{ MANAGEMENT_RULE : follows
    CANTEEN ||--|{ MENU : creates
    MENU ||--|{ MENU_ITEM : contains
    MENU_ITEM }|--|{ DISH : is
    DISH ||--|{ DISH_INGREDIENT : uses
    DISH_INGREDIENT }|--|{ INGREDIENT : is
    INGREDIENT ||--|{ NUTRITION_INFO : has
    CANTEEN ||--|{ LEDGER : maintains
    LEDGER ||--|{ LEDGER_RECORD : contains
    LEDGER_RECORD ||--o{ EMPLOYEE : recorded_by
    CANTEEN ||--|{ PURCHASE_PLAN : makes
    PURCHASE_PLAN ||--|{ PURCHASE_ORDER : generates
    PURCHASE_ORDER ||--o{ SUPPLIER : sent_to
    PURCHASE_ORDER ||--|{ PURCHASE_ITEM : contains
    PURCHASE_ITEM }|--|{ INGREDIENT : is
```

```
PURCHASE_ORDER ||--o{ DELIVERY : leads_to
DELIVERY ||--o{ INSPECTION : requires
INSPECTION ||--|{ INSPECTION_ITEM : contains
INSPECTION_ITEM }|--|{ INGREDIENT : is
INSPECTION ||--o{ STOCK_IN : results_in
STOCK_IN ||--|{ STOCK_IN_ITEM : contains
STOCK_IN_ITEM }|--|{ INGREDIENT : is
STOCK_IN ||--o{ BATCH : creates
BATCH ||--|{ INVENTORY : updates
INVENTORY ||--o{ INGREDIENT : tracks
INVENTORY ||--o{ STOCK_OUT : leads_to
STOCK_OUT ||--|{ STOCK_OUT_ITEM : contains
STOCK_OUT_ITEM }|--|{ INGREDIENT : is
STOCK_OUT ||--o{ EMPLOYEE : requested_by
PURCHASE_ORDER ||--o{ SETTLEMENT : requires
SETTLEMENT ||--o{ SUPPLIER : pays_to
CANTEEN ||--|{ COST_ACCOUNTING : performs
COST_ACCOUNTING ||--o{ REVENUE : includes
COST_ACCOUNTING ||--o{ EXPENSE : includes
CANTEEN ||--|{ RISK_ASSESSMENT : undergoes
CANTEEN ||--|{ TRACEABILITY_INFO : generates
CANTEEN ||--|{ RATING : receives
USER ||--o{ NOTIFICATION : receives
USER ||--o{ COMPLAINT : submits
COMPLAINT ||--o{ SUPPLIER : related_to
CANTEEN ||--o{ CANTEEN_STYLE : displays
KNOWLEDGE_BASE ||--|{ KB_ITEM : contains
CITY_DATA_CENTER ||--o{ CANTEEN : monitors
```

核心实体说明:

- **USER:** 系统用户 (管理员, 食堂员工, 监管人员等)
- **ROLE:** 用户角色
- **PERMISSION:** 权限
- **SCHOOL:** 学校
- **CANTEEN:** 食堂
- **EMPLOYEE:** 食堂从业人员
- **HEALTH_CERT:** 健康证
- **SUPPLIER:** 供应商
- **INGREDIENT:** 食材
- **WASTE_RECYCLER:** 厨余回收企业
- **CANTEEN_INFO:** 食堂基本信息
- **LICENSE:** 相关证照
- **MANAGEMENT_RULE:** 管理制度
- **MENU:** 食谱
- **DISH:** 菜品
- **NUTRITION_INFO:** 营养信息

- **LEDGER:** 台账
- **LEDGER_RECORD:** 台账记录
- **PURCHASE_PLAN:** 采购计划
- **PURCHASE_ORDER:** 采购订单
- **DELIVERY:** 送货单
- **INSPECTION:** 验收单
- **STOCK_IN:** 入库单
- **BATCH:** 食材批次
- **INVENTORY:** 库存
- **STOCK_OUT:** 出库单
- **SETTLEMENT:** 结算单
- **COST_ACCOUNTING:** 成本核算
- **RISK_ASSESSMENT:** 风险评估
- **TRACEABILITY_INFO:** 溯源信息
- **RATING:** 规范化评分
- **NOTIFICATION:** 通知公告
- **COMPLAINT:** 投诉建议
- **CANTEEN_STYLE:** 食堂风采
- **KNOWLEDGE_BASE:** 知识库
- **CITY_DATA_CENTER:** 市级数据中心 (逻辑概念，可能涉及接口交互)

4.3 逻辑模型设计 (部分核心表)

以下列出部分核心表的逻辑结构设计，省略了部分通用字段（如创建时间、更新时间、创建人、更新人、逻辑删除标记等）。

用户信息表 (t_user)

字段名	类型	约束	描述
user_id	BIGINT	PK	用户ID
username	VARCHAR(50)	UQ, NN	用户名/登录账号
password	VARCHAR(100)	NN	加密后的密码
nickname	VARCHAR(50)		昵称
phone	VARCHAR(20)	UQ	手机号
email	VARCHAR(100)	UQ	邮箱
avatar_url	VARCHAR(255)		头像URL

字段名	类型	约束	描述
user_type	TINYINT	NN	用户类型 (1:学校管理员, 2:食堂员工, 3:监管人员, 4:供应商, 5:系统管理员)
school_id	BIGINT	FK(t_school)	所属学校ID (用户类型为1,2时)
canteen_id	BIGINT	FK(t_canteen)	所属食堂ID (用户类型为2时)
supplier_id	BIGINT	FK(t_supplier)	所属供应商ID (用户类型为4时)
status	TINYINT	NN	状态 (0:禁用, 1:启用)
last_login_time	DATETIME		最后登录时间

角色信息表 (t_role)

字段名	类型	约束	描述
role_id	BIGINT	PK	角色ID
role_name	VARCHAR(50)	UQ, NN	角色名称
role_key	VARCHAR(50)	UQ, NN	角色标识符
description	VARCHAR(255)		描述
status	TINYINT	NN	状态 (0:禁用, 1:启用)

用户角色关联表 (t_user_role)

字段名	类型	约束	描述
id	BIGINT	PK	关联ID
user_id	BIGINT	FK(t_user), NN	用户ID
role_id	BIGINT	FK(t_role), NN	角色ID

权限信息表 (t_permission)

字段名	类型	约束	描述
permission_id	BIGINT	PK	权限ID

字段名	类型	约束	描述
permission_name	VARCHAR(100)	NN	权限名称
permission_key	VARCHAR(100)	UQ, NN	权限标识符 (如: sys:user:add)
parent_id	BIGINT		父权限ID (用于菜单/按钮层级)
type	TINYINT	NN	类型 (1:菜单, 2:按钮/API)
path	VARCHAR(255)		路由地址 (菜单类型)
component	VARCHAR(255)		组件路径 (菜单类型)
icon	VARCHAR(100)		图标
sort_order	INT		排序
status	TINYINT	NN	状态 (0:禁用, 1:启用)

角色权限关联表 (t_role_permission)

字段名	类型	约束	描述
id	BIGINT	PK	关联ID
role_id	BIGINT	FK(t_role), NN	角色ID
permission_id	BIGINT	FK(t_permission), NN	权限ID

学校信息表 (t_school)

字段名	类型	约束	描述
school_id	BIGINT	PK	学校ID
school_name	VARCHAR(100)	UQ, NN	学校名称
address	VARCHAR(255)		地址
contact_person	VARCHAR(50)		联系人
contact_phone	VARCHAR(20)		联系电话
district	VARCHAR(50)		所属区县

字段名	类型	约束	描述
status	TINYINT	NN	状态 (0:禁用, 1:启用)

食堂信息表 (t_canteen)

字段名	类型	约束	描述
canteen_id	BIGINT	PK	食堂ID
school_id	BIGINT	FK(t_school), NN	所属学校ID
canteen_name	VARCHAR(100)	NN	食堂名称
address	VARCHAR(255)		地址
manager_id	BIGINT	FK(t_user)	食堂负责人ID
status	TINYINT	NN	状态 (0:禁用, 1:启用)

食材信息表 (t_ingredient)

字段名	类型	约束	描述
ingredient_id	BIGINT	PK	食材ID
ingredient_name	VARCHAR(100)	UQ, NN	食材名称
category_id	BIGINT	FK(t_ingredient_category)	分类ID
unit	VARCHAR(20)		单位 (如: kg, g, 个)
specification	VARCHAR(100)		规格
description	TEXT		描述
nutrition_info	JSON		营养成分 (能量, 蛋白质, 脂肪等)
is_built_in	BOOLEAN	NN	是否内置 (系统预置)
status	TINYINT	NN	状态 (0:禁用, 1:启用)

菜品信息表 (t_dish)

字段名	类型	约束	描述
dish_id	BIGINT	PK	菜品ID
dish_name	VARCHAR(100)	NN	菜品名称
category_id	BIGINT	FK(t_dish_category)	分类ID (主食, 荤菜, 素菜等)
description	TEXT		描述
image_url	VARCHAR(255)		图片URL
canteen_id	BIGINT	FK(t_canteen)	所属食堂ID (食堂自定义菜品)
is_common	BOOLEAN		是否常用菜品
status	TINYINT	NN	状态 (0:禁用, 1:启用)

菜品食材关联表 (t_dish_ingredient)

字段名	类型	约束	描述
id	BIGINT	PK	关联ID
dish_id	BIGINT	FK(t_dish), NN	菜品ID
ingredient_id	BIGINT	FK(t_ingredient), NN	食材ID
quantity	DECIMAL(10,2)	NN	标准用量
unit	VARCHAR(20)	NN	用量单位
crowd_type	TINYINT		适用人群类型 (可选)

食谱信息表 (t_menu)

字段名	类型	约束	描述
menu_id	BIGINT	PK	食谱ID
canteen_id	BIGINT	FK(t_canteen), NN	食堂ID
menu_date	DATE	NN	食谱日期
meal_time	TINYINT	NN	餐次 (1:早餐, 2:午餐, 3:晚餐)

字段名	类型	约束	描述
status	TINYINT	NN	状态 (0:草稿, 1:已发布)

食谱菜品关联表 (t_menu_item)

字段名	类型	约束	描述
id	BIGINT	PK	关联ID
menu_id	BIGINT	FK(t_menu), NN	食谱ID
dish_id	BIGINT	FK(t_dish), NN	菜品ID
estimated_quantity	DECIMAL(10,2)		预计份数

台账记录表 (t_ledger_record)

字段名	类型	约束	描述
record_id	BIGINT	PK	记录ID
canteen_id	BIGINT	FK(t_canteen), NN	食堂ID
ledger_type_id	BIGINT	FK(t_ledger_type), NN	台账类型ID (晨检, 消毒等)
record_time	DATETIME	NN	记录时间
recorder_id	BIGINT	FK(t_user), NN	记录人ID
content	JSON		记录内容 (根据台账类型定义)
photos	JSON		相关照片URL列表
status	TINYINT	NN	状态 (0:待完成, 1:已完成)
remark	VARCHAR(255)		备注

采购订单表 (t_purchase_order)

字段名	类型	约束	描述
order_id	BIGINT	PK	订单ID

字段名	类型	约束	描述
order_sn	VARCHAR(50)	UQ, NN	订单编号
canteen_id	BIGINT	FK(t_canteen), NN	采购食堂ID
supplier_id	BIGINT	FK(t_supplier), NN	供应商ID
order_type	TINYINT	NN	订单类型 (1:线上, 2:线下补录)
total_amount	DECIMAL(12,2)		订单总金额
order_time	DATETIME	NN	下单时间
expected_delivery_time	DATETIME		期望送达时间
actual_delivery_time	DATETIME		实际送达时间
status	TINYINT	NN	订单状态 (待确认, 待发货, 待验收, 已完成, 已取消)
remark	VARCHAR(255)		备注

库存信息表 (t_inventory)

字段名	类型	约束	描述
inventory_id	BIGINT	PK	库存ID
canteen_id	BIGINT	FK(t_canteen), NN	食堂ID
ingredient_id	BIGINT	FK(t_ingredient), NN	食材ID
batch_id	BIGINT	FK(t_batch)	批次ID (可选, 用于批次管理)
quantity	DECIMAL(10,2)	NN	当前库存数量
unit	VARCHAR(20)	NN	单位

字段名	类型	约束	描述
warning_threshold	DECIMAL(10,2)		库存预警阈值
last_update_time	DATETIME	NN	最后更新时间

4.4 物理模型设计

- **数据库引擎**：统一使用 InnoDB 引擎，支持事务和行级锁。
- **字符集**：统一使用 utf8mb4 字符集，支持更广泛的字符，包括 Emoji。
- **索引策略**：
 - 主键使用自增ID。
 - 为经常用于查询条件的字段（如 user_id , canteen_id , order_sn , record_time 等）创建索引。
 - 为外键关联字段创建索引。
 - 对于组合查询频繁的字段，创建联合索引。
 - 避免对大文本字段（TEXT, JSON）直接创建索引，考虑使用全文索引或冗余字段。
- **分区策略**：对于数据量可能非常大的表（如 t_ledger_record , t_purchase_order , t_inventory_log 等），考虑按时间（如按月或按季度）进行分区，提高查询和维护效率。
- **命名规范**：
 - 表名：小写字母 + 下划线，加 t_ 前缀 (e.g., t_user , t_purchase_order)。
 - 字段名：小写字母 + 下划线 (e.g., user_id , order_time)。
 - 索引名：idx_ + 表名（去前缀）+ 字段名 (e.g., idx_user_username)。
 - 外键名：fk_ + 当前表名（去前缀）+ 关联表名（去前缀） (e.g., fk_user_role_user)。

4.5 数据字典

详细的数据字典将在详细设计阶段提供，包含所有表的字段定义、数据类型、约束、枚举值含义等。

4.6 数据库维护与备份

- **备份策略**：
 - 每日全量备份。
 - 每小时增量备份（或按需配置）。
 - 备份数据异地存储。
- **维护计划**：
 - 定期进行数据库性能监控和优化。

- 定期清理过期数据和日志。
- 定期进行数据库版本升级和补丁更新。

5. 接口设计

5.1 接口设计原则

智慧食堂项目的接口设计遵循以下原则：

1. **RESTful 风格**：采用 REST 架构风格，使用 HTTP 方法（GET、POST、PUT、DELETE 等）表示操作语义。
2. **统一响应格式**：所有接口返回统一的 JSON 格式，包含状态码、消息和数据。
3. **版本控制**：在 URL 中包含版本号，便于接口升级和兼容性维护。
4. **权限控制**：基于 RBAC 模型，对接口进行权限控制。
5. **参数验证**：严格验证请求参数，确保数据安全和有效性。
6. **文档完备**：使用 Swagger 自动生成接口文档，保持文档与代码的一致性。
7. **错误处理**：统一的错误处理机制，提供明确的错误码和错误信息。
8. **幂等性**：确保关键操作的幂等性，防止重复提交导致的数据异常。

5.2 接口分类

系统接口按照功能模块和使用场景分为以下几类：

1. **认证授权接口**：用户登录、注销、刷新令牌等。
2. **用户管理接口**：用户信息管理、角色权限管理等。
3. **基础信息接口**：学校、食堂、从业人员、供应商等基础信息管理。
4. **食谱管理接口**：食材库、菜品库、食谱编辑等。
5. **台账管理接口**：台账配置、记录、查询、统计等。
6. **采购管理接口**：采购计划、订单、入库、出库、库存等。
7. **统计分析接口**：采购统计、综合统计、风险评估等。
8. **数据中心接口**：市级数据接入、信息调阅等。
9. **系统管理接口**：通知公告、食堂风采、停餐报备等。
10. **文件管理接口**：文件上传、下载等。
11. **微信小程序接口**：专用于小程序的接口。

5.3 接口规范

5.3.1 请求规范

URL 格式：

```
https://{domain}/api/v{version}/{module}/{resource}/{id}
```

示例：

```
https://smart-canteen.example.com/api/v1/menu/dishes/123
```

HTTP 方法：

- GET：查询资源
- POST：创建资源
- PUT：更新资源（全量更新）
- PATCH：更新资源（部分更新）
- DELETE：删除资源

请求头：

```
Content-Type: application/json
Authorization: Bearer {token}
X-Request-ID: {requestId}
```

请求参数：

- 路径参数：作为 URL 的一部分，如 `/users/{userId}`
- 查询参数：附加在 URL 后面，如 `/users?page=1&size=10`
- 请求体：JSON 格式的数据

5.3.2 响应规范

响应状态码：

- 200 OK：请求成功
- 201 Created：资源创建成功
- 204 No Content：请求成功但无返回内容
- 400 Bad Request：请求参数错误
- 401 Unauthorized：未认证
- 403 Forbidden：无权限
- 404 Not Found：资源不存在
- 500 Internal Server Error：服务器内部错误

响应体格式：

```
{
  "code": 200,
  "message": "操作成功",
  "data": {
```

```
// 具体数据
}
}
```

分页响应格式：

```
{
  "code": 200,
  "message": "操作成功",
  "data": {
    "total": 100,
    "pages": 10,
    "current": 1,
    "size": 10,
    "records": [
      // 数据列表
    ]
  }
}
```

错误响应格式：

```
{
  "code": 400,
  "message": "参数错误",
  "data": null,
  "errors": [
    {
      "field": "username",
      "message": "用户名不能为空"
    }
  ]
}
```

5.4 核心接口定义

以下列出系统的部分核心接口定义，完整接口将在详细设计阶段提供。

5.4.1 认证授权接口

用户登录

- URL: /api/v1/auth/login
- 方法: POST
- 请求体:
json

```
{
  "username": "admin",
  "password": "password",
  "loginType": "account" // account: 账号密码, wxmp: 微信小程序
}
- 响应体:
json
{
  "code": 200,
  "message": "登录成功",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresIn": 3600,
    "userInfo": {
      "userId": 1,
      "username": "admin",
      "nickname": "管理员",
      "avatar": "https://example.com/avatar.jpg",
      "roles": ["admin"]
    }
  }
}
```

刷新令牌

```
- URL: /api/v1/auth/refresh-token
- 方法: POST
- 请求体:
json
{
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
- 响应体:
json
{
  "code": 200,
  "message": "刷新成功",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",

```

```
"expiresIn": 3600
```

```
}
```

```
}
```

用户注销

- URL: /api/v1/auth/logout

- 方法: POST

- 请求头: Authorization: Bearer {token}

- 响应体:

```
json
```

```
{
```

```
"code": 200,
```

```
"message": "注销成功",
```

```
"data": null
```

```
}
```

5.4.2 食谱管理接口

获取食谱列表

- URL: /api/v1/menu/list

- 方法: GET

- 参数:

- canteenId: 食堂ID

- startDate: 开始日期

- endDate: 结束日期

- page: 页码

- size: 每页大小

- 响应体:

```
json
```

```
{
```

```
"code": 200,
```

```
"message": "操作成功",
```

```
"data": {
```

```
"total": 30,
```

```
"pages": 3,
```

```
"current": 1,
```

```
"size": 10,
```

```
"records": [
```

```
{
```

```
"menuId": 1,
```

```
"canteenId": 1,
```

```
"menuDate": "2025-06-01",
"mealTime": 2,
"mealTimeName": "午餐",
"status": 1,
"statusName": "已发布",
"dishes": [
  {
    "dishId": 101,
    "dishName": "红烧排骨",
    "category": "荤菜",
    "imageUrl": "https://example.com/images/paigu.jpg"
  },
  // 更多菜品...
]
},
// 更多食谱...
]
```

创建/编辑食谱

- URL: /api/v1/menu/save

- 方法: POST

- 请求体:

```
json
{
  "menuId": null, // null表示新建，非null表示编辑
  "canteenId": 1,
  "menuDate": "2025-06-01",
  "mealTime": 2,
  "dishes": [
    {
      "dishId": 101,
      "estimatedQuantity": 100
    },
    {
      "dishId": 102,
      "estimatedQuantity": 80
    }
  ],
  // 更多菜品...
}
```

```
]
}
- 响应体:
json
{
  "code": 200,
  "message": "保存成功",
  "data": {
    "menuId": 1
  }
}
```

获取菜品库

```
- URL: /api/v1/dish/list
- 方法: GET
- 参数:
  - canteenId: 食堂ID
  - categoryId: 分类ID
  - keyword: 关键词
  - page: 页码
  - size: 每页大小
- 响应体:
json
{
  "code": 200,
  "message": "操作成功",
  "data": {
    "total": 200,
    "pages": 20,
    "current": 1,
    "size": 10,
    "records": [
      {
        "dishId": 101,
        "dishName": "红烧排骨",
        "categoryId": 2,
        "categoryName": "荤菜",
        "description": "经典红烧排骨",
        "imageUrl": "https://example.com/images/paigu.jpg",
        "isCommon": true,
```

```
"ingredients": [  
  {  
    "ingredientId": 1001,  
    "ingredientName": "猪排骨",  
    "quantity": 0.5,  
    "unit": "kg"  
  },  
  // 更多食材...  
]  
},  
// 更多菜品...  
]  
}  
}
```

5.4.3 台账管理接口

获取台账记录列表

- URL: /api/v1/ledger/records
- 方法: GET
- 参数:
 - canteenId: 食堂ID
 - ledgerTypeId: 台账类型ID
 - startDate: 开始日期
 - endDate: 结束日期
 - status: 状态
 - page: 页码
 - size: 每页大小
- 响应体:

```
json  
{  
  "code": 200,  
  "message": "操作成功",  
  "data": {  
    "total": 100,  
    "pages": 10,  
    "current": 1,  
    "size": 10,  
    "records": [  
      {
```

```
"recordId": 1,  
"canteenId": 1,  
"ledgerTypeId": 1,  
"ledgerTypeName": "人员晨检",  
"recordTime": "2025-06-01 08:00:00",  
"recorderName": "张三",  
"content": {  
  "temperature": 36.5,  
  "healthStatus": "正常",  
  "remarks": ""  
},  
"photos": [  
  "https://example.com/photos/check_1.jpg"  
],  
"status": 1,  
"statusName": "已完成"  
},  
// 更多记录...  
]  
}  
}
```

提交台账记录

- URL: /api/v1/ledger/record/submit

- 方法: POST

- 请求体:

```
json  
{  
  "recordId": null, // null表示新建，非null表示编辑  
  "canteenId": 1,  
  "ledgerTypeId": 1,  
  "recordTime": "2025-06-01 08:00:00",  
  "content": {  
    "temperature": 36.5,  
    "healthStatus": "正常",  
    "remarks": ""  
  },  
  "photos": [  
    "https://example.com/photos/check_1.jpg"  
  ]  
}
```

```
}  
- 响应体:  
json  
{  
  "code": 200,  
  "message": "提交成功",  
  "data": {  
    "recordId": 1  
  }  
}
```

5.4.4 采购管理接口

创建采购订单

```
- URL: /api/v1/purchase/order/create  
- 方法: POST  
- 请求体:  
json  
{  
  "canteenId": 1,  
  "supplierId": 1,  
  "orderType": 1,  
  "expectedDeliveryTime": "2025-06-02 10:00:00",  
  "items": [  
    {  
      "ingredientId": 1001,  
      "quantity": 10,  
      "unit": "kg",  
      "unitPrice": 15.5  
    },  
    {  
      "ingredientId": 1002,  
      "quantity": 5,  
      "unit": "kg",  
      "unitPrice": 8.0  
    }  
  ],  
  // 更多商品...  
  "remark": "请保证新鲜度"  
}
```

- 响应体:

```
json
{
  "code": 200,
  "message": "创建成功",
  "data": {
    "orderId": 1,
    "orderSn": "PO202506010001"
  }
}
```

获取库存列表

- URL: /api/v1/inventory/list

- 方法: GET

- 参数:

- canteenId: 食堂ID

- categoryId: 分类ID

- keyword: 关键词

- warningFlag: 是否预警

- page: 页码

- size: 每页大小

- 响应体:

```
json
{
  "code": 200,
  "message": "操作成功",
  "data": {
    "total": 50,
    "pages": 5,
    "current": 1,
    "size": 10,
    "records": [
      {
        "inventoryId": 1,
        "canteenId": 1,
        "ingredientId": 1001,
        "ingredientName": "猪排骨",
        "categoryName": "肉类",
        "quantity": 15.5,
        "unit": "kg",

```

```
    "warningThreshold": 5.0,  
    "isWarning": false,  
    "lastUpdateTime": "2025-06-01 14:30:00"  
  },  
  // 更多库存...  
]  
}  
}
```

5.5 外部接口

5.5.1 微信小程序接口

微信授权登录

- URL: /api/v1/wxmp/auth/login

- 方法: POST

- 请求体:

```
json  
{  
  "code": "wx_code",  
  "userInfo": {  
    "nickName": "微信用户",  
    "avatarUrl": "https://thirdwx.qlogo.cn/..."  
  }  
}
```

- 响应体:

```
json  
{  
  "code": 200,  
  "message": "登录成功",  
  "data": {  
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
    "expiresIn": 3600,  
    "userInfo": {  
      "userId": 1,  
      "username": "wx_user_123",  
      "nickname": "微信用户",  
      "avatar": "https://thirdwx.qlogo.cn/...",  
      "roles": ["canteen_staff"]  
    }  
  }  
}
```

```
}  
}  
}
```

5.5.2 市级数据中心接口

数据上报接口

- URL: /api/v1/datacenter/report

- 方法: POST

- 请求体:

```
json  
{  
  "schoolId": 1,  
  "dataType": "menu", // menu, ledger, purchase, etc.  
  "dataContent": {  
    // 具体数据内容，根据dataType不同而不同  
  },  
  "reportTime": "2025-06-01 18:00:00"  
}
```

- 响应体:

```
json  
{  
  "code": 200,  
  "message": "上报成功",  
  "data": {  
    "reportId": "R202506010001"  
  }  
}
```

数据同步接口

- URL: /api/v1/datacenter/sync

- 方法: GET

- 参数:

- schoolId: 学校ID

- dataType: 数据类型

- lastSyncTime: 上次同步时间

- 响应体:

```
json  
{  
  "code": 200,  
  "message": "同步成功",
```

```
"data": {  
  "syncTime": "2025-06-01 18:30:00",  
  "dataList": [  
    // 同步的数据列表  
  ]  
}
```

5.6 接口安全

5.6.1 认证机制

系统采用基于 JWT (JSON Web Token) 的认证机制：

- 令牌生成**：用户登录成功后，服务器生成包含用户身份信息的 JWT 令牌。
- 令牌传递**：客户端在后续请求中通过 Authorization 头传递令牌。
- 令牌验证**：服务器验证令牌的有效性和完整性。
- 令牌刷新**：令牌过期前，客户端可使用刷新令牌获取新的访问令牌。

5.6.2 授权控制

系统基于 RBAC 模型实现细粒度的授权控制：

- 角色定义**：预定义多种角色（如系统管理员、学校管理员、食堂员工等）。
- 权限分配**：为每个角色分配相应的权限。
- 接口保护**：每个接口都定义所需的权限，未授权的请求将被拒绝。
- 数据过滤**：根据用户角色和所属组织，自动过滤数据访问范围。

5.6.3 接口防护

为保障接口安全，系统采取以下措施：

- 参数验证**：严格验证请求参数的类型、格式和范围。
- 防重放攻击**：请求中包含时间戳和唯一标识，防止请求被重放。
- 防SQL注入**：使用参数化查询和ORM框架，防止SQL注入攻击。
- 防XSS攻击**：对输入输出进行HTML转义，防止跨站脚本攻击。
- 接口限流**：对频繁访问的接口实施限流措施，防止恶意请求。
- 日志审计**：记录关键接口的访问日志，便于安全审计和问题排查。

5.7 接口文档

系统使用 Swagger 自动生成接口文档，包含以下内容：

- 接口描述**：接口的功能说明和使用场景。

- 2. **请求方法**：HTTP方法（GET、POST等）。
- 3. **请求URL**：接口的完整URL路径。
- 4. **请求参数**：包括路径参数、查询参数和请求体参数，以及每个参数的类型、是否必须、默认值和说明。
- 5. **响应格式**：包括成功响应和各种错误响应的格式和示例。
- 6. **权限要求**：访问接口所需的权限。
- 7. **示例代码**：调用接口的示例代码。

接口文档将通过专门的文档页面提供给开发人员和测试人员，并保持与实际代码的同步更新。

6. 安全设计

6.1 安全设计原则

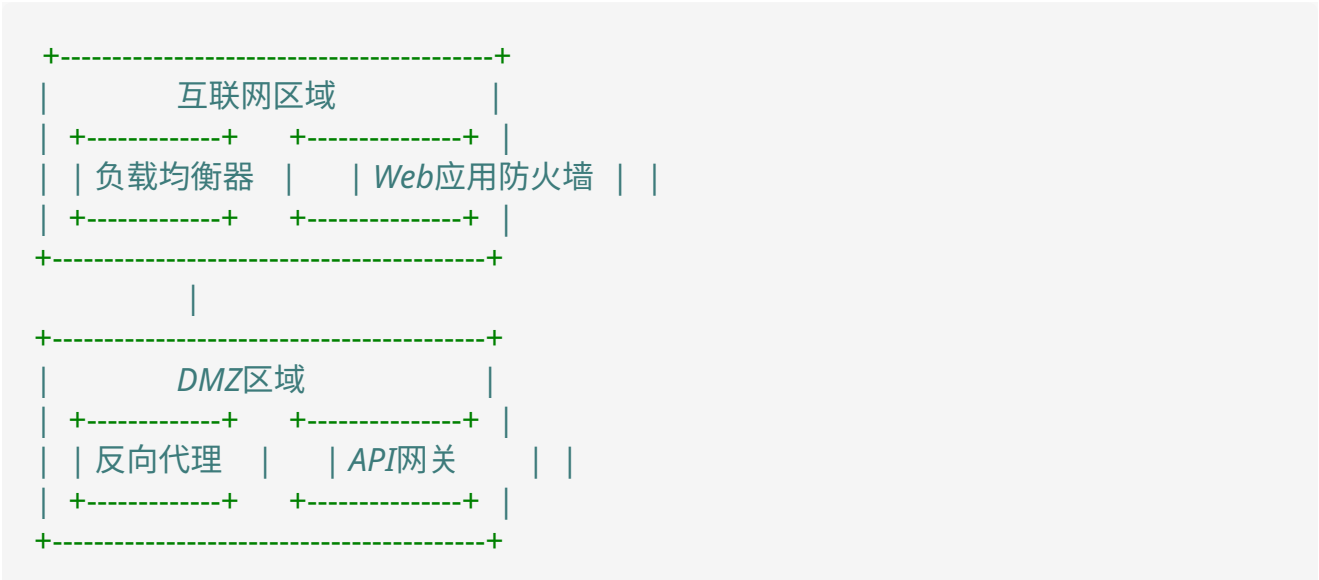
智慧食堂项目的安全设计遵循以下原则：

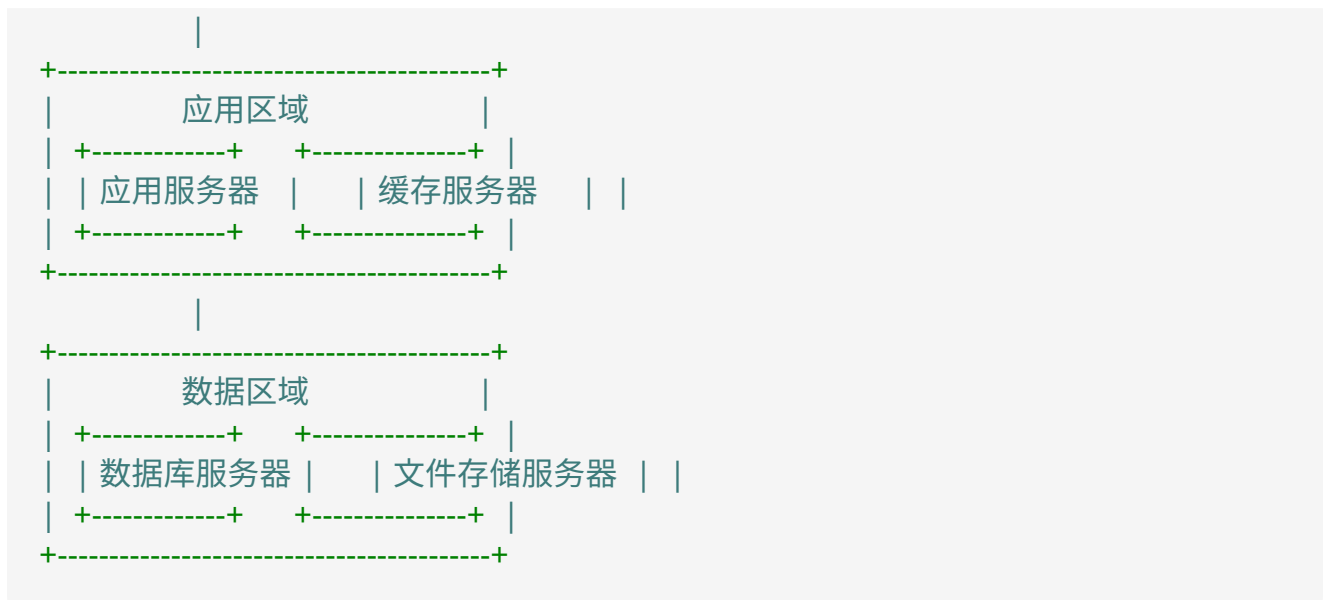
- 1. **纵深防御**：采用多层次、多方面的安全防护措施，构建全方位的安全防线。
- 2. **最小权限**：用户和系统组件只被授予完成其任务所需的最小权限。
- 3. **安全默认配置**：系统默认配置应当是安全的，避免因配置不当导致安全风险。
- 4. **数据保护**：对敏感数据进行加密存储和传输，防止未授权访问和泄露。
- 5. **完整性保护**：确保数据和系统组件的完整性，防止未授权修改。
- 6. **可审计性**：记录关键操作和安全事件，支持安全审计和事件追溯。
- 7. **持续更新**：定期更新安全措施，应对新出现的安全威胁。
- 8. **安全意识**：提高用户和管理员的安全意识，减少人为安全风险。

6.2 网络安全

6.2.1 网络架构

系统采用分层网络架构，通过网络隔离和访问控制提高安全性：





6.2.2 网络防护措施

1. 防火墙：

2. 部署边界防火墙，控制进出网络的流量。
3. 配置内部防火墙，隔离不同安全区域。
4. 实施严格的访问控制策略，只允许必要的网络连接。

5. Web应用防火墙（WAF）：

6. 部署WAF，防御常见的Web攻击，如SQL注入、XSS、CSRF等。
7. 配置自定义规则，针对特定应用的安全需求。
8. 实时监控和阻断可疑请求。

9. 入侵检测/防御系统（IDS/IPS）：

10. 部署IDS/IPS，监控网络流量，检测和阻止异常行为。
11. 配置规则库，识别已知的攻击模式。
12. 定期更新规则库，应对新的攻击方式。

13. DDoS防护：

14. 采用云服务提供商的DDoS防护服务。
15. 配置流量清洗策略，过滤恶意流量。
16. 制定DDoS应急响应计划。

17. 网络隔离：

18. 使用VLAN划分不同网络区域。
19. 实施网络访问控制列表（ACL）。

20. 采用微分段技术，限制横向移动。

21. 安全通信：

22. 全站HTTPS，使用TLS 1.2+加密所有通信。

23. 配置安全的密码套件，禁用不安全的加密算法。

24. 实施HTTP安全头，如HSTS、CSP等。

6.3 应用安全

6.3.1 身份认证

1. 多因素认证：

2. 支持账号密码、短信验证码、微信扫码等多种认证方式。

3. 对关键操作要求二次认证。

4. 管理员账号强制使用多因素认证。

5. 密码策略：

6. 强制使用复杂密码（包含大小写字母、数字和特殊字符）。

7. 定期密码更换（90天）。

8. 密码历史记录（禁止使用最近5次使用过的密码）。

9. 密码尝试限制（5次失败后锁定账号）。

10. 会话管理：

11. 基于JWT的无状态会话管理。

12. 会话超时设置（Web端30分钟，移动端7天）。

13. 会话固定防护。

14. 并发会话控制（同一账号限制登录设备数）。

6.3.2 访问控制

1. 基于角色的访问控制（RBAC）：

2. 预定义角色（系统管理员、学校管理员、食堂员工等）。

3. 细粒度权限分配。

4. 权限继承和组合机制。

5. 数据访问控制：

6. 行级权限控制，用户只能访问其所属组织的数据。

7. 字段级权限控制，敏感字段对特定角色可见。

8. 数据脱敏，对敏感信息进行显示脱敏。

9. API访问控制：

10. 基于OAuth 2.0的API授权。

11. API密钥管理。

12. API访问频率限制。

6.3.3 输入验证与输出编码

1. 输入验证：

2. 客户端和服务端双重验证。

3. 白名单验证策略。

4. 数据类型、格式、长度、范围验证。

5. 特殊字符过滤。

6. 输出编码：

7. HTML实体编码，防止XSS攻击。

8. URL编码，防止注入攻击。

9. JSON编码，防止JSON注入。

10. SQL参数化查询，防止SQL注入。

6.3.4 安全开发实践

1. 安全编码规范：

2. 制定并遵循安全编码规范。

3. 代码审查，重点关注安全问题。

4. 使用安全的第三方库和框架。

5. 安全测试：

6. 静态应用安全测试（SAST）。

7. 动态应用安全测试（DAST）。

8. 渗透测试。

9. 安全漏洞扫描。

10. 依赖管理：

11. 使用安全的依赖版本。

12. 定期更新依赖，修复已知漏洞。

13. 依赖漏洞扫描。

6.4 数据安全

6.4.1 数据分类

根据敏感程度，将系统数据分为以下几类：

1. **公开数据**：可公开访问的数据，如食堂风采、通知公告等。
2. **内部数据**：仅组织内部可访问的数据，如食谱、台账记录等。
3. **敏感数据**：需要特殊保护的数据，如采购价格、成本核算等。
4. **高敏感数据**：需要最高级别保护的数据，如用户密码、健康信息等。

6.4.2 数据加密

1. **传输加密**：
 2. 使用TLS 1.2+加密所有网络通信。
 3. 配置安全的密码套件。
 4. 证书管理和更新机制。
5. **存储加密**：
 6. 敏感数据字段加密存储（如密码使用bcrypt/PBKDF2等算法加密）。
 7. 数据库透明加密（TDE）。
 8. 文件存储加密。
9. **密钥管理**：
 10. 密钥生成和分发机制。
 11. 密钥轮换策略。
 12. 密钥备份和恢复机制。

6.4.3 数据备份与恢复

1. **备份策略**：
 2. 定期全量备份（每日）。
 3. 增量备份（每小时）。
 4. 备份数据加密存储。
 5. 异地备份。
6. **恢复机制**：
 7. 定义恢复点目标（RPO）和恢复时间目标（RTO）。
 8. 制定数据恢复流程。
 9. 定期测试恢复流程。

6.4.4 数据销毁

1. **数据留存策略：**
2. 定义不同类型数据的留存期限。
3. 自动归档和清理机制。
4. **安全销毁：**
5. 数据安全擦除方法。
6. 物理介质销毁流程。
7. 第三方服务终止后的数据处理。

6.5 运维安全

6.5.1 安全配置管理

1. **基线配置：**
2. 服务器安全基线配置。
3. 数据库安全基线配置。
4. 应用服务器安全基线配置。
5. 网络设备安全基线配置。
6. **配置审计：**
7. 定期审计系统配置。
8. 自动化配置检查。
9. 配置偏差报告和修复。

6.5.2 日志与监控

1. **日志管理：**
2. 集中式日志收集（ELK Stack）。
3. 关键操作日志记录。
4. 安全事件日志记录。
5. 日志保留策略（至少6个月）。
6. **安全监控：**
7. 实时安全事件监控。
8. 异常行为检测。
9. 安全告警机制。
10. 安全仪表盘。

11. 漏洞管理：

12. 定期漏洞扫描。
13. 漏洞评估和分级。
14. 漏洞修复流程。
15. 漏洞修复验证。

6.5.3 安全响应

1. 安全事件响应计划：

2. 定义安全事件类型和级别。
3. 制定响应流程和责任分工。
4. 上报和通知机制。
5. 事件处理和恢复流程。

6. 应急演练：

7. 定期进行安全应急演练。
8. 演练评估和改进。
9. 更新应急响应计划。

6.6 合规性

6.6.1 法规遵从

系统设计和实施需遵循以下法规和标准：

1. 《中华人民共和国网络安全法》
2. 《中华人民共和国数据安全法》
3. 《中华人民共和国个人信息保护法》
4. 《信息安全技术 个人信息安全规范》（GB/T 35273-2020）
5. 《信息安全技术 网络安全等级保护基本要求》（GB/T 22239-2019）

6.6.2 安全评估

1. 等级保护测评：

2. 按照网络安全等级保护要求进行测评。
3. 整改测评发现的问题。
4. 定期复测。

5. 安全审计：

6. 定期进行内部安全审计。

- 7. 必要时进行第三方安全审计。
- 8. 审计发现问题的跟踪和整改。

6.7 安全培训与意识

- 1. 安全培训计划：
- 2. 开发人员安全培训。
- 3. 运维人员安全培训。
- 4. 用户安全意识培训。
- 5. 安全文档：
- 6. 安全操作手册。
- 7. 安全事件响应指南。
- 8. 用户安全使用指南。
- 9. 安全意识提升：
- 10. 定期安全通报。
- 11. 安全提示和最佳实践分享。
- 12. 安全意识测试。

7. 部署方案

7.1 部署环境要求

7.1.1 硬件要求

生产环境推荐配置：

服务器类型	数量	CPU	内存	存储	网络
应用服务器	2+	8核+	16GB+	100GB+ SSD	千兆网卡
数据库服务器	2	8核+	32GB+	500GB+ SSD (RAID 10)	千兆网卡
缓存服务器	2	4核+	8GB+	50GB+ SSD	千兆网卡
文件存储服务器	1	4核+	8GB+	1TB+ (可扩展)	千兆网卡
负载均衡器	2	4核+	8GB+	50GB+ SSD	千兆网卡

测试环境推荐配置：

服务器类型	数量	CPU	内存	存储	网络
应用服务器	1	4核+	8GB+	100GB+ SSD	千兆网卡
数据库服务器	1	4核+	16GB+	200GB+ SSD	千兆网卡
缓存服务器	1	2核+	4GB+	50GB+ SSD	千兆网卡

开发环境推荐配置：

服务器类型	数量	CPU	内存	存储	网络
综合服务器	1	4核+	16GB+	200GB+ SSD	千兆网卡

7.1.2 软件要求

操作系统：

- CentOS 7.x / 8.x 或 Ubuntu 18.04 LTS / 20.04 LTS
- 内核版本 4.x 以上

容器平台：

- Docker 20.10.x 以上
- Kubernetes 1.20.x 以上 (生产环境)

数据库：

- MySQL 8.0.x
- Redis 6.x

中间件：

- Nginx 1.20.x 以上
- RabbitMQ 3.8.x 以上

运行时环境：

- JDK 11 / 17
- Node.js 14.x 以上

监控工具：

- Prometheus + Grafana
- ELK Stack (Elasticsearch, Logstash, Kibana)

7.1.3 网络要求

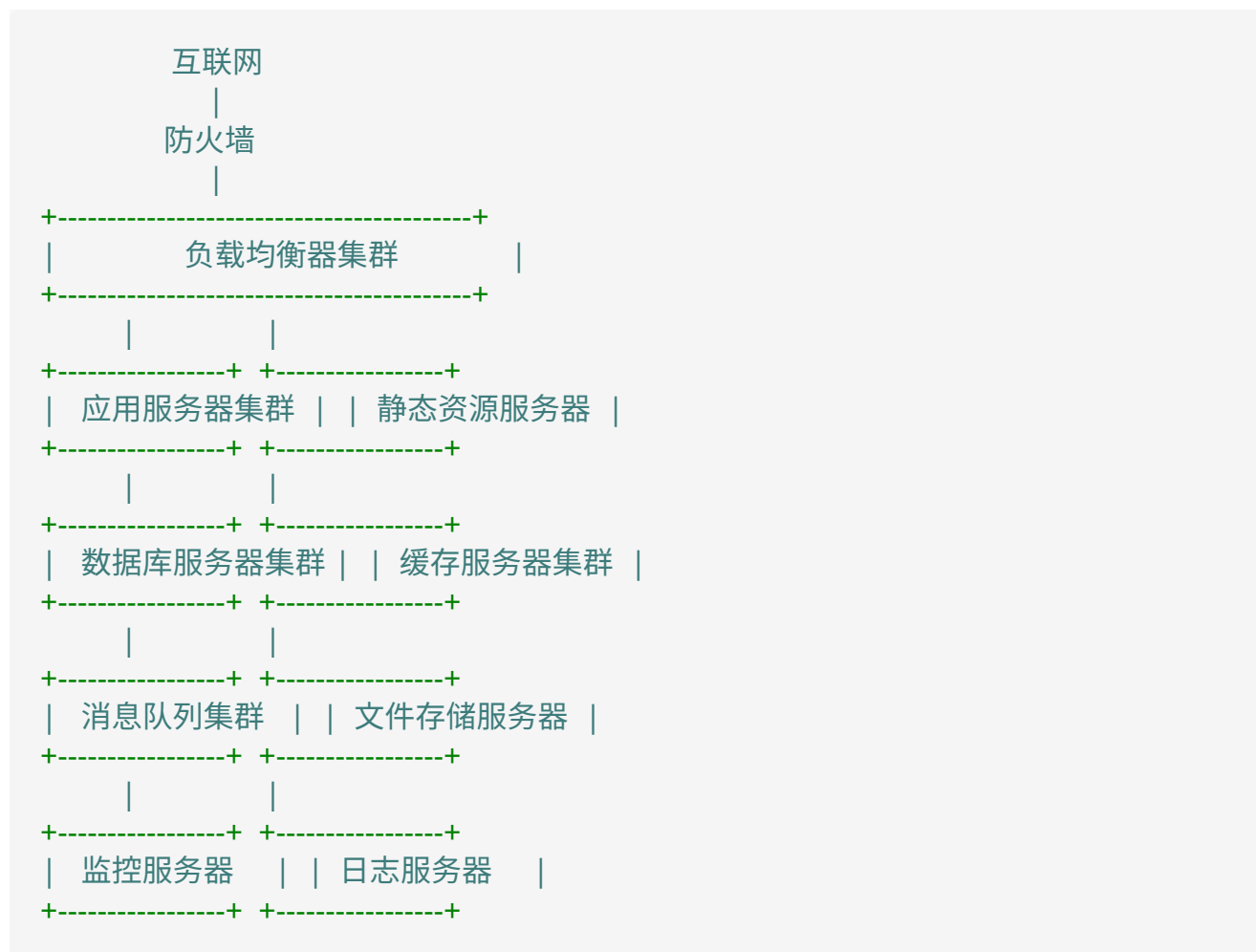
- 内网带宽：千兆以上
- 外网带宽：50Mbps 以上（视并发用户数调整）

- 固定公网 IP
- 域名及 SSL 证书
- 防火墙配置：开放必要端口（80, 443, 22 等）

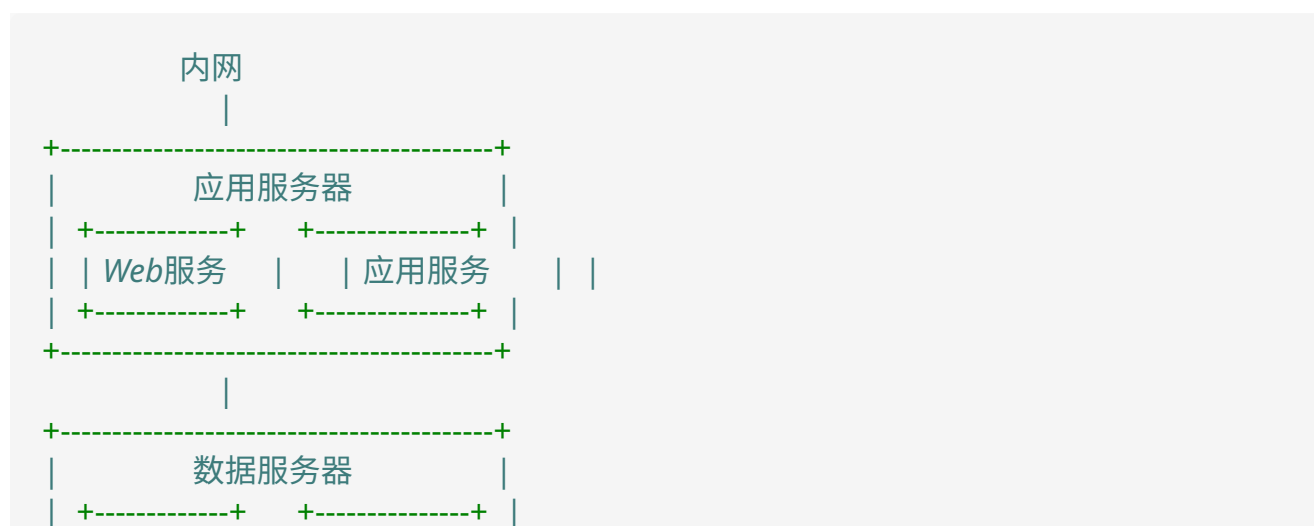
7.2 部署架构

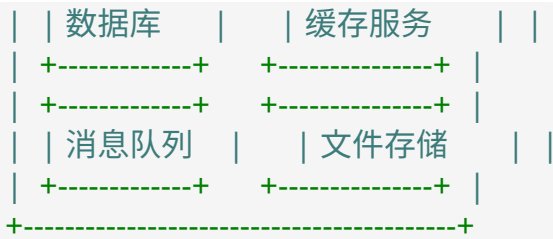
7.2.1 物理部署架构

生产环境：



测试/开发环境：





7.2.2 逻辑部署架构

容器化部署：



7.3 部署流程

7.3.1 环境准备

- 1. 基础设施准备：
- 2. 服务器硬件配置和上架
- 3. 网络配置（IP分配、防火墙规则等）
- 4. 操作系统安装和基础配置
- 5. 软件环境准备：
- 6. 安装Docker和Kubernetes（生产环境）

7. 安装基础软件包和依赖
8. 配置系统参数（如最大文件描述符、内核参数等）
9. **存储准备：**
10. 配置数据库存储（如RAID、LVM等）
11. 配置文件存储（如NFS、对象存储等）
12. 配置备份存储
13. **网络准备：**
14. 配置内部网络（如VLAN、子网等）
15. 配置外部网络（如负载均衡、DNS等）
16. 配置SSL证书

7.3.2 应用部署

1. **数据库部署：**
2. 安装MySQL数据库
3. 配置主从复制（生产环境）
4. 初始化数据库结构和基础数据
5. 配置数据库参数和优化
6. **中间件部署：**
7. 部署Redis缓存集群
8. 部署RabbitMQ消息队列
9. 部署Nginx反向代理
10. 配置各中间件参数
11. **应用服务部署：**
12. 构建应用Docker镜像
13. 部署Kubernetes资源（Deployment、Service、ConfigMap等）
14. 配置应用参数和环境变量
15. 部署前端静态资源
16. **监控系统部署：**
17. 部署Prometheus和Grafana
18. 部署ELK Stack
19. 配置监控告警规则

20. 配置日志收集和分析

7.3.3 部署验证

1. 功能验证：

2. 验证各模块功能是否正常
3. 验证系统集成是否正常
4. 验证数据流转是否正常

5. 性能验证：

6. 进行负载测试
7. 验证系统响应时间
8. 验证系统并发处理能力

9. 安全验证：

10. 进行安全扫描
11. 验证权限控制是否有效
12. 验证数据加密是否正常

13. 可用性验证：

14. 进行故障模拟测试
15. 验证高可用机制是否有效
16. 验证数据备份和恢复是否正常

7.4 高可用方案

7.4.1 应用层高可用

1. 负载均衡：

2. 使用Nginx或云服务提供商的负载均衡服务
3. 配置健康检查机制
4. 实现会话保持（如有状态服务）

5. 应用集群：

6. 部署多个应用实例
7. 使用Kubernetes进行容器编排和自动恢复
8. 配置水平自动扩展（HPA）

9. 服务降级与熔断：

10. 使用Spring Cloud Circuit Breaker实现服务熔断
11. 配置服务降级策略
12. 实现请求限流机制

7.4.2 数据层高可用

1. 数据库高可用：

2. MySQL主从复制
3. 配置自动故障转移
4. 定期数据备份

5. 缓存高可用：

6. Redis集群或哨兵模式
7. 多节点数据分片
8. 配置持久化策略

9. 消息队列高可用：

10. RabbitMQ集群
11. 配置镜像队列
12. 实现消息持久化

13. 文件存储高可用：

14. 使用分布式文件系统或对象存储
15. 配置数据冗余和备份
16. 实现跨区域复制（可选）

7.4.3 网络层高可用

1. 多链路冗余：

2. 配置多条网络链路
3. 实现自动路由切换
4. 配置链路负载均衡

5. DNS高可用：

6. 使用多个DNS服务提供商
7. 配置DNS轮询或故障转移

8. 合理设置TTL值

9. **防火墙高可用：**

10. 部署冗余防火墙设备

11. 配置状态同步

12. 实现自动故障转移

7.5 扩展性方案

7.5.1 水平扩展

1. **应用层扩展：**

2. 增加应用服务器节点

3. 配置自动扩缩容策略

4. 优化负载均衡算法

5. **数据库扩展：**

6. 实现读写分离

7. 配置数据分片

8. 使用数据库中间件（如ShardingSphere）

9. **缓存扩展：**

10. 增加缓存节点

11. 优化缓存策略

12. 实现多级缓存

7.5.2 垂直扩展

1. **资源升级：**

2. 增加服务器CPU和内存

3. 升级存储设备

4. 优化网络设备

5. **性能优化：**

6. 代码优化

7. SQL优化

8. JVM参数调优

9. 操作系统参数调优

7.6 灾备方案

7.6.1 数据备份

1. **备份策略：**
2. 每日全量备份
3. 定时增量备份
4. 事务日志备份
5. **备份存储：**
6. 本地备份
7. 异地备份
8. 云存储备份
9. **备份验证：**
10. 定期备份恢复测试
11. 备份数据完整性检查
12. 备份过程监控

7.6.2 灾难恢复

1. **恢复流程：**
2. 制定详细的恢复流程文档
3. 明确恢复优先级
4. 定义恢复时间目标（RTO）和恢复点目标（RPO）
5. **灾备环境：**
6. 配置灾备数据中心（可选）
7. 实现数据实时或准实时同步
8. 配置自动或手动切换机制
9. **应急预案：**
10. 制定各类灾难的应急预案
11. 定期进行灾难恢复演练
12. 不断优化和完善预案

7.7 运维管理

7.7.1 监控告警

1. **系统监控：**
2. 服务器资源监控（CPU、内存、磁盘、网络等）
3. 应用性能监控（响应时间、吞吐量、错误率等）
4. 数据库监控（连接数、查询性能、锁等）
5. 网络监控（带宽、延迟、丢包率等）
6. **告警机制：**
7. 设置多级告警阈值
8. 配置多渠道告警通知（邮件、短信、微信等）
9. 实现告警升级机制
10. 配置告警抑制和聚合规则
11. **日志管理：**
12. 集中式日志收集
13. 日志分析和可视化
14. 异常日志告警
15. 日志存储和归档策略

7.7.2 变更管理

1. **版本发布：**
2. 制定版本发布流程
3. 实现蓝绿部署或金丝雀发布
4. 配置自动化部署流水线
5. 制定回滚策略
6. **配置管理：**
7. 使用配置中心统一管理配置
8. 实现配置版本控制
9. 配置变更审计
10. 敏感配置加密存储
11. **资源管理：**
12. 服务器资源管理

- 13. 容器资源管理
- 14. 存储资源管理
- 15. 网络资源管理

7.7.3 运维自动化

- 1. 自动化工具：
 - 2. 使用Ansible等工具实现自动化配置管理
 - 3. 使用Jenkins等工具实现自动化构建和部署
 - 4. 使用Terraform等工具实现基础设施即代码（IaC）
- 5. 自动化脚本：
 - 6. 开发运维自动化脚本
 - 7. 实现常见运维任务自动化
 - 8. 配置定时任务和触发器
- 9. 自动化测试：
 - 10. 实现自动化功能测试
 - 11. 实现自动化性能测试
 - 12. 实现自动化安全测试

7.8 部署清单

7.8.1 软件清单

软件名称	版本	用途	安装方式
CentOS/Ubuntu	7.x/8.x 或 18.04/20.04	操作系统	物理安装或虚拟机
Docker	20.10.x+	容器运行时	包管理器或官方脚本
Kubernetes	1.20.x+	容器编排	kubeadm
MySQL	8.0.x	关系型数据库	Docker或包管理器
Redis	6.x	缓存服务	Docker或包管理器
RabbitMQ	3.8.x+	消息队列	Docker或包管理器
Nginx	1.20.x+	反向代理和负载均衡	Docker或包管理器
JDK	11/17	Java运行环境	Docker或包管理器

软件名称	版本	用途	安装方式
Node.js	14.x+	前端构建环境	Docker或包管理器
Prometheus	2.x	监控系统	Docker或包管理器
Grafana	8.x+	监控可视化	Docker或包管理器
Elasticsearch	7.x+	日志存储和搜索	Docker或包管理器
Logstash	7.x+	日志收集和处理	Docker或包管理器
Kibana	7.x+	日志可视化	Docker或包管理器

7.8.2 配置清单

配置项	说明	配置文件位置
应用配置	应用服务的配置参数	/opt/app/config/
数据库配置	MySQL数据库配置	/etc/mysql/my.cnf
Redis配置	Redis缓存配置	/etc/redis/redis.conf
RabbitMQ配置	消息队列配置	/etc/rabbitmq/rabbitmq.conf
Nginx配置	Web服务器配置	/etc/nginx/nginx.conf
Kubernetes配置	容器编排配置	/etc/kubernetes/
监控配置	Prometheus和Grafana配置	/etc/prometheus/, /etc/grafana/
日志配置	ELK Stack配置	/etc/elasticsearch/, /etc/logstash/, /etc/kibana/

7.8.3 文档清单

文档名称	说明	存放位置
部署指南	详细的部署步骤和说明	/docs/deployment/
配置手册	各组件的配置参数说明	/docs/configuration/
运维手册	日常运维操作指南	/docs/operation/

文档名称	说明	存放位置
故障处理手册	常见故障的处理方法	/docs/troubleshooting/
备份恢复手册	数据备份和恢复操作指南	/docs/backup/
监控告警手册	监控系统使用和告警处理指南	/docs/monitoring/
安全加固手册	系统安全加固指南	/docs/security/

7.9 部署风险与应对策略

风险	可能影响	应对策略
硬件故障	系统不可用	1. 使用冗余硬件 2. 配置故障自动转移 3. 准备备用设备
网络故障	系统访问中断	1. 配置多条网络链路 2. 实施网络监控 3. 制定网络应急预案
数据丢失	业务数据损坏或丢失	1. 实施定期备份 2. 配置数据冗余 3. 测试数据恢复流程
安全漏洞	系统被入侵或数据泄露	1. 定期安全扫描 2. 及时更新补丁 3. 实施安全加固
性能瓶颈	系统响应缓慢	1. 进行性能测试 2. 优化系统配置 3. 准备扩容方案
部署失败	系统无法正常启动	1. 制定详细部署计划 2. 进行预部署测试 3. 准备回滚方案
人为操作错误	系统配置错误或数据损坏	1. 制定操作规范 2. 实施权限控制 3. 进行操作培训